

第十章 优化

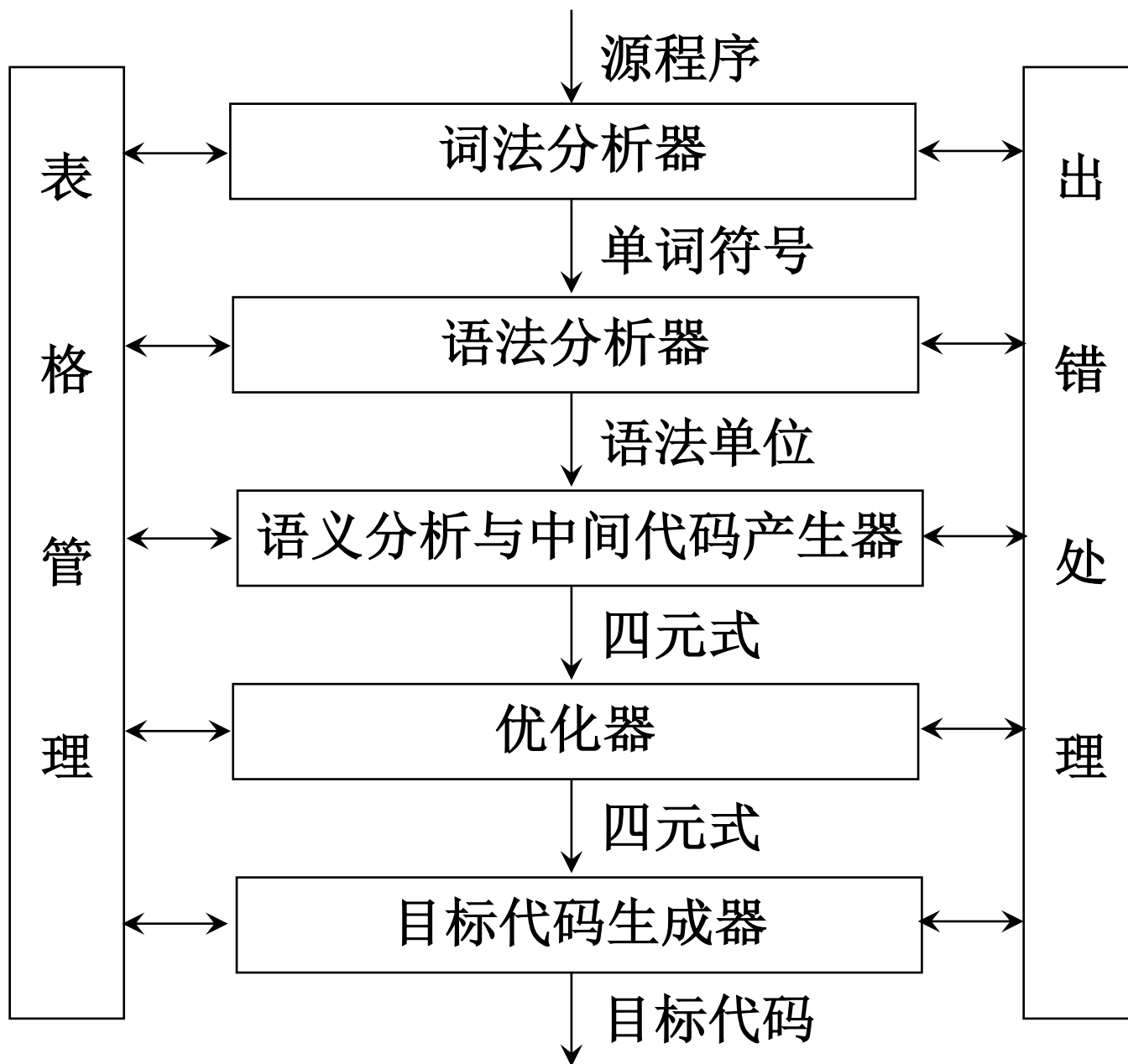


图1.1 编译程序总框

10.1 概述

- 优化：对程序进行各种等价变换，使得从变换后的程序出发，能生成更有效的目标代码。
 - 等价：不应改变程序运行的结果
 - 有效：目标代码运行时间较短，占用的存储空间较小
 - 合算：以较低的代价取得较好的优化成果

- 常用的优化技术：
 - 1) 删除公共子表达式
 - 2) 复写传播
 - 3) 删除无用代码
 - 4) 代码外提
 - 5) 强度削弱
 - 6) 删除归纳变量

```

void quicksort (m, n);
int m, n;
{
    int i, j;
    int v, x;
    if (n<=m) return;
    /* fragment begins here */
    i=m-1; j=n; v=a[n];
    while (1) {
        do i=i+1; while (a[i]<v);
        do j=j-1; while (a[j]>v);
        if (i>=j) break;
        x=a[i]; a[i]=a[j]; a[j]=x;
    }
    x=a[i]; a[i]=a[n]; a[n]=x;
    /* fragment ends here */
    quicksort (m, j); quicksort (i+1, n);
}

```

- 基本块(p279)

指程序中一顺序执行的语句序列，其中只有一个入口和一个出口，入口是其中的第一个语句，出口是其中的最后一个语句。

执行时，只能从其入口进入，从其出口退出。

例，基本块：

$t_1 := a * a$

$t_2 := a * b$

$t_3 := 2 * t_2$

$t_4 := t_1 + t_3$

$t_5 := b * b$

$t_6 := t_4 + t_5$

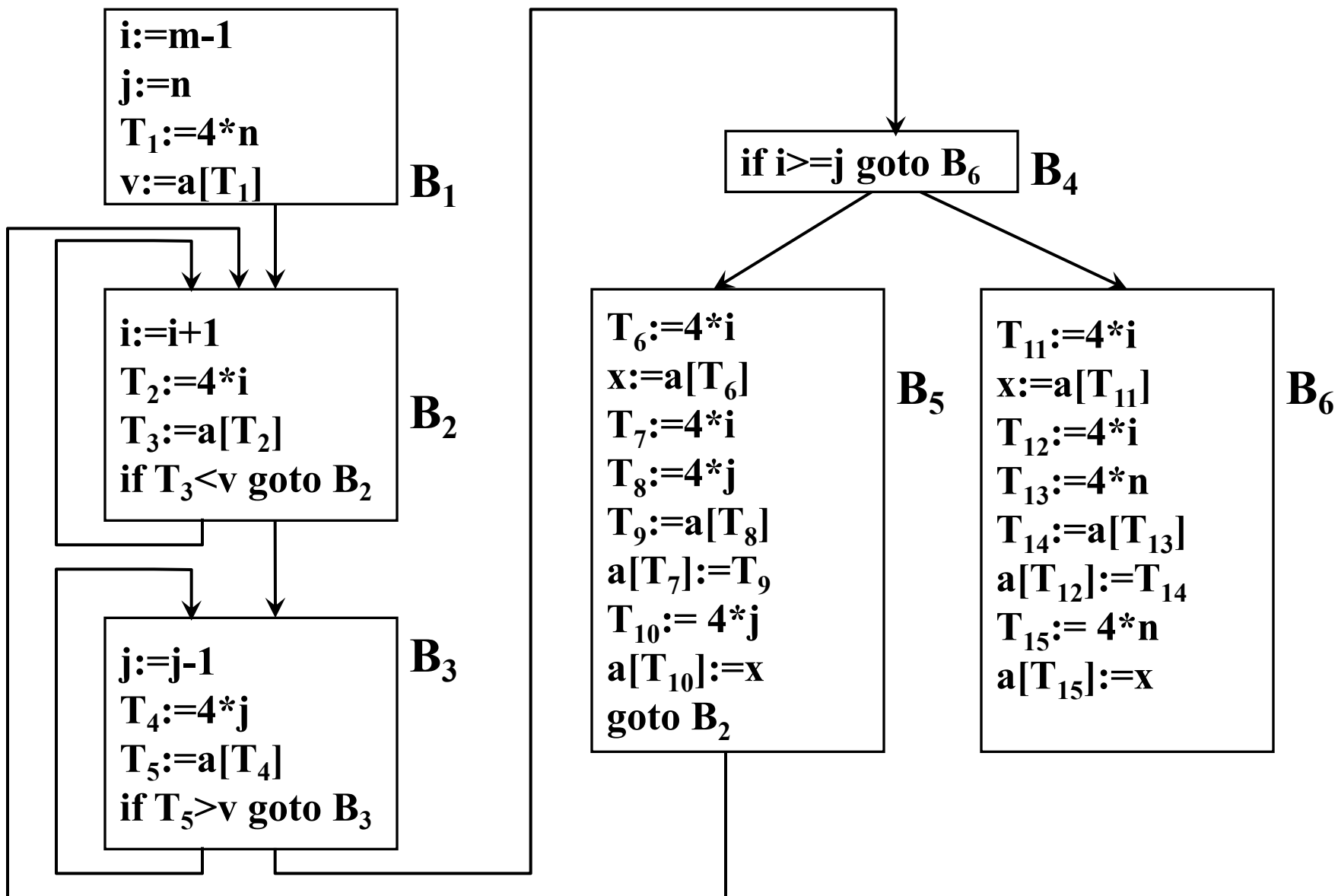


图10.2 中间代码程序段

- 删除公共子表达式
 - 如果一个表达式 E 在前面已计算过，并且在这之后 E 中变量的值没有改变，则称 E 为公共子表达式。
 - 对于公共子表达式，可以避免对它的重复计算，称为删除公共子表达式（删除多余运算）。

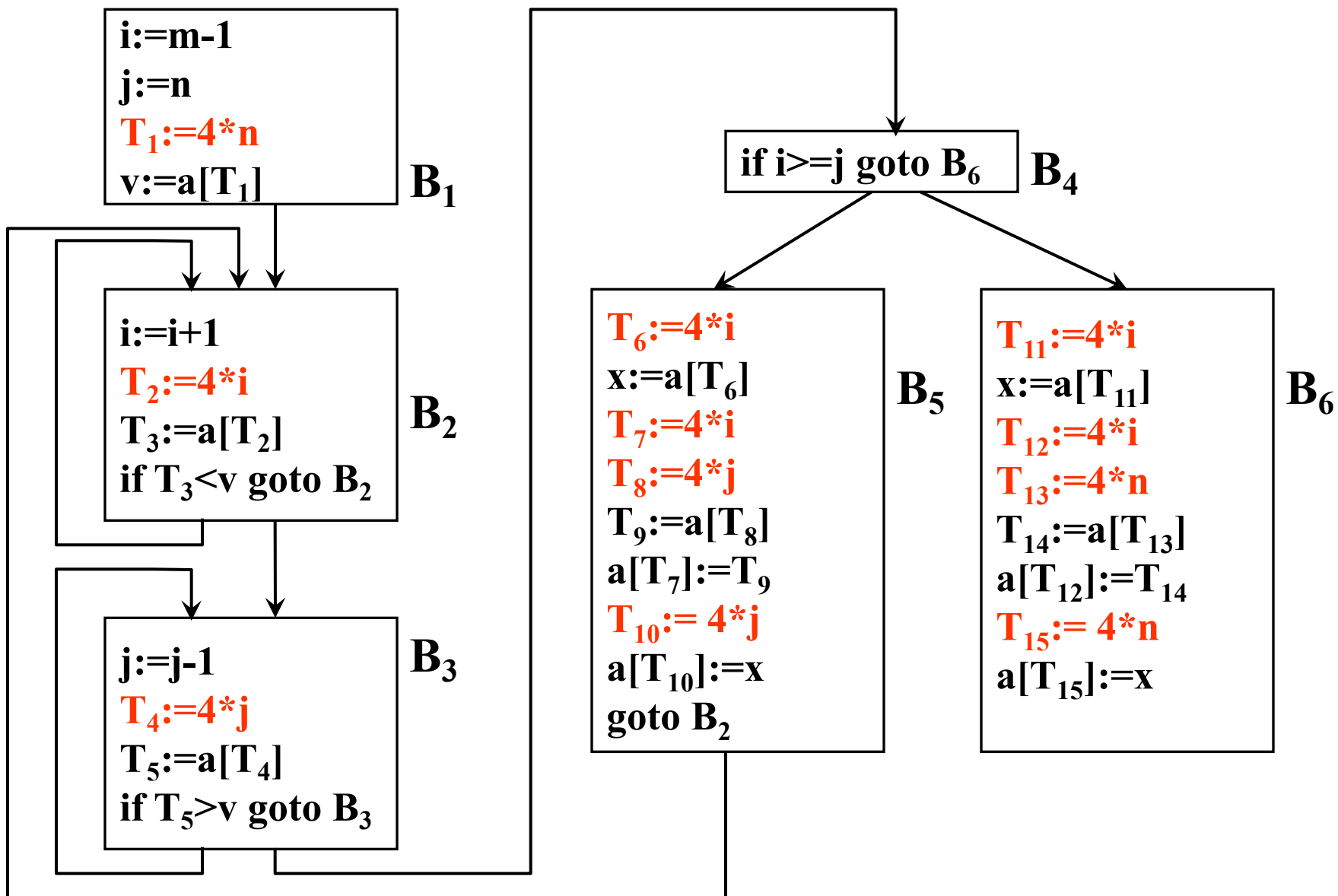


图10.2 中间代码程序段

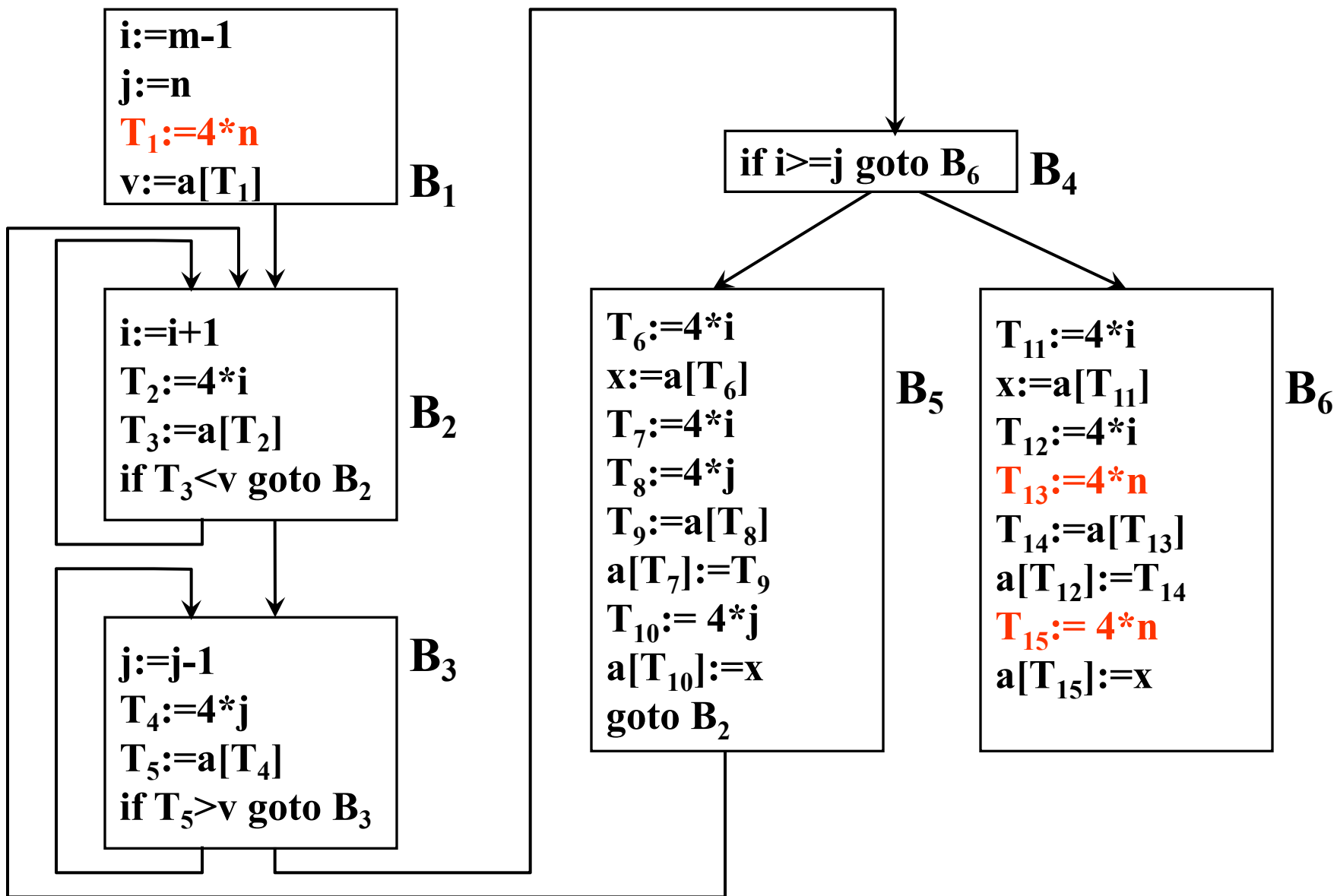


图10.2 中间代码程序段

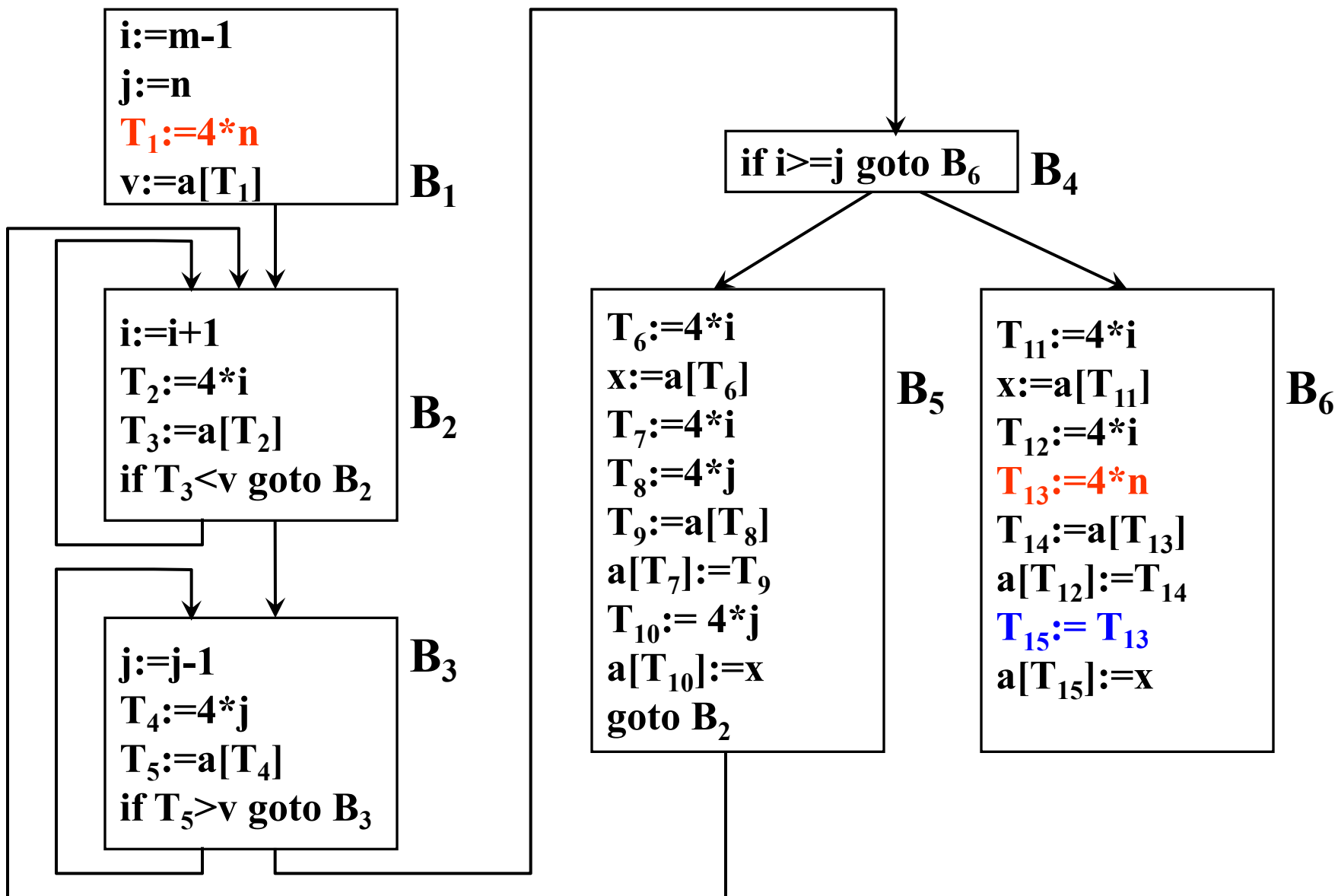


图10.2 中间代码程序段

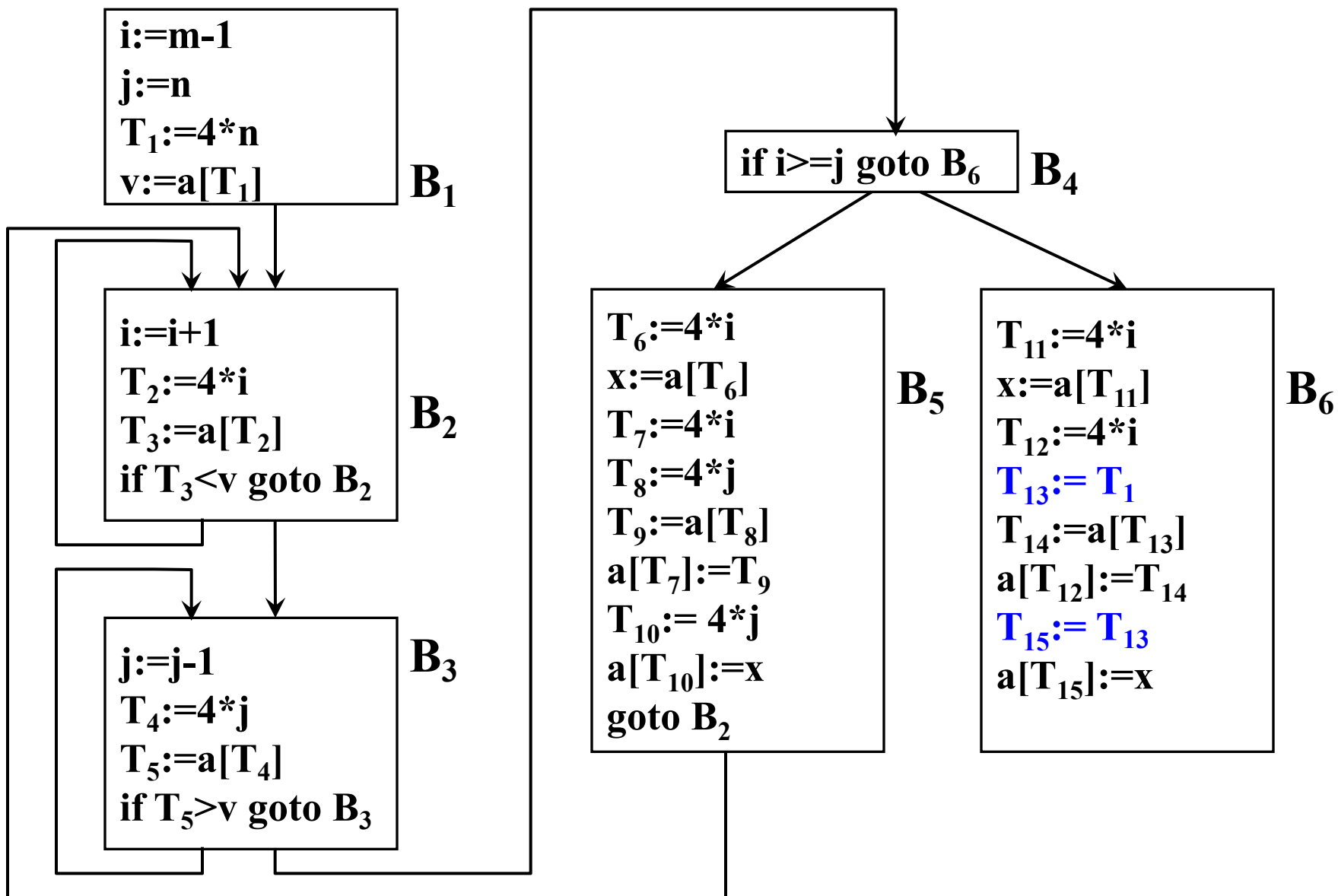


图10.3 删除公共子表达式

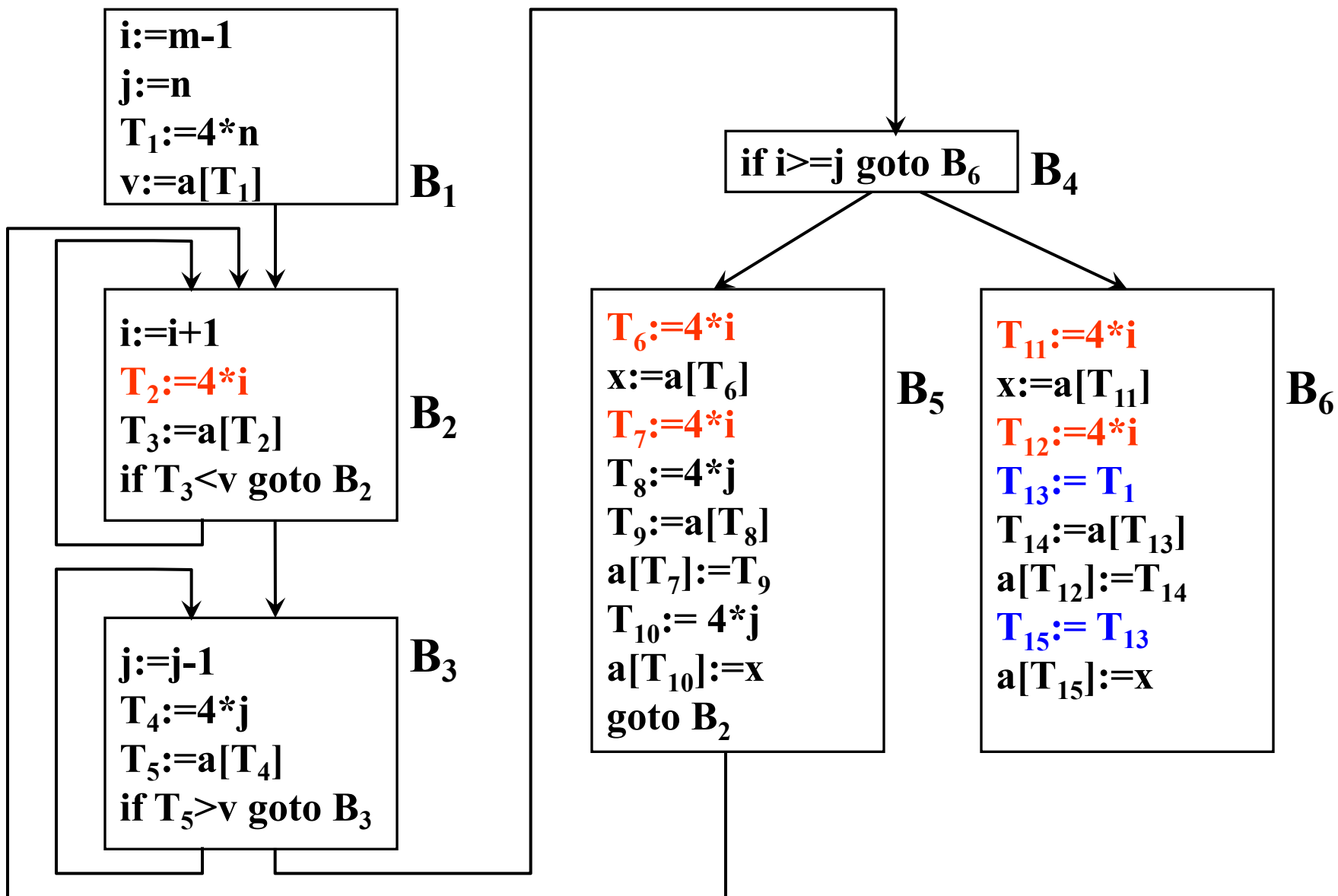


图10.3 删除公共子表达式

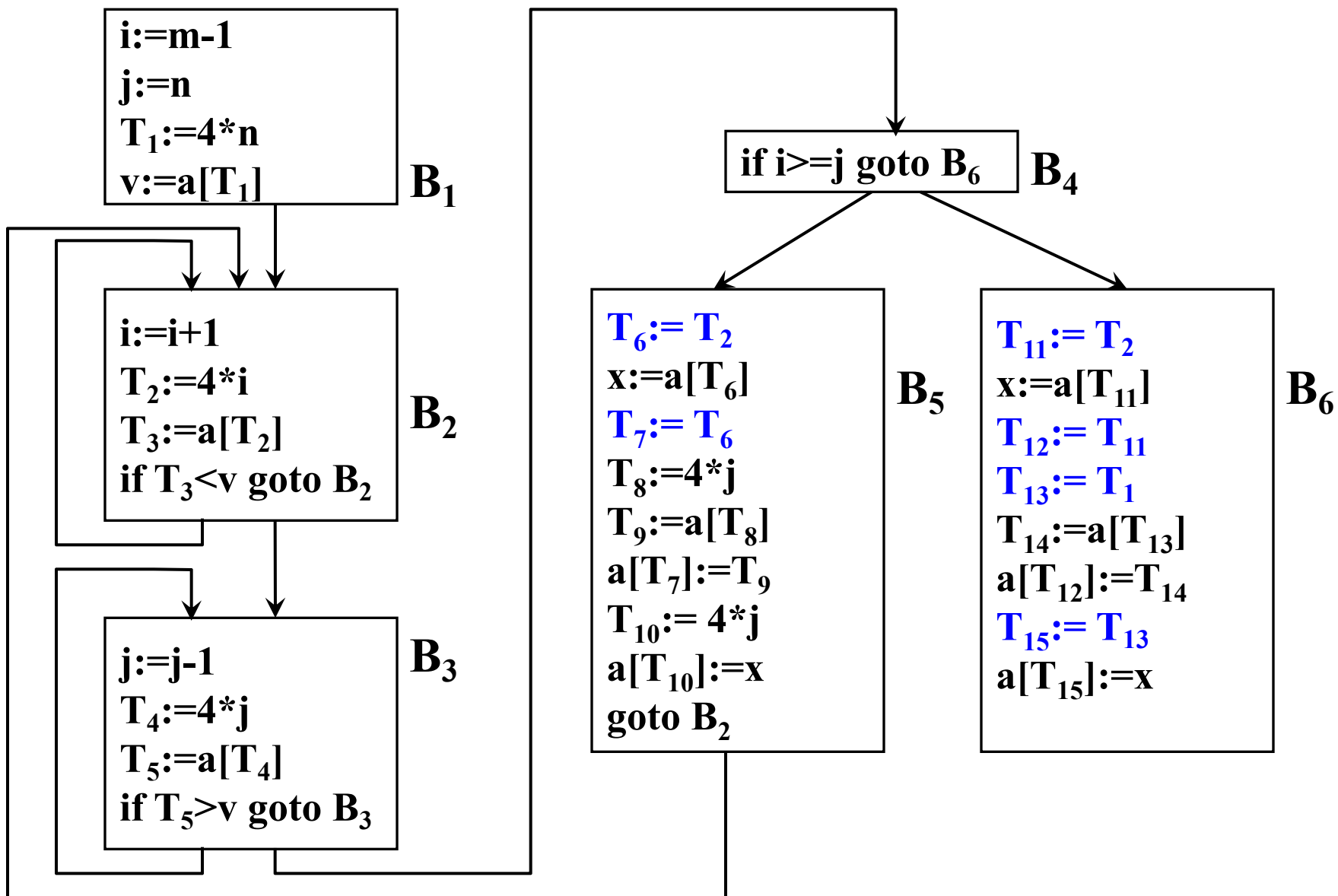


图10.3 删除公共子表达式

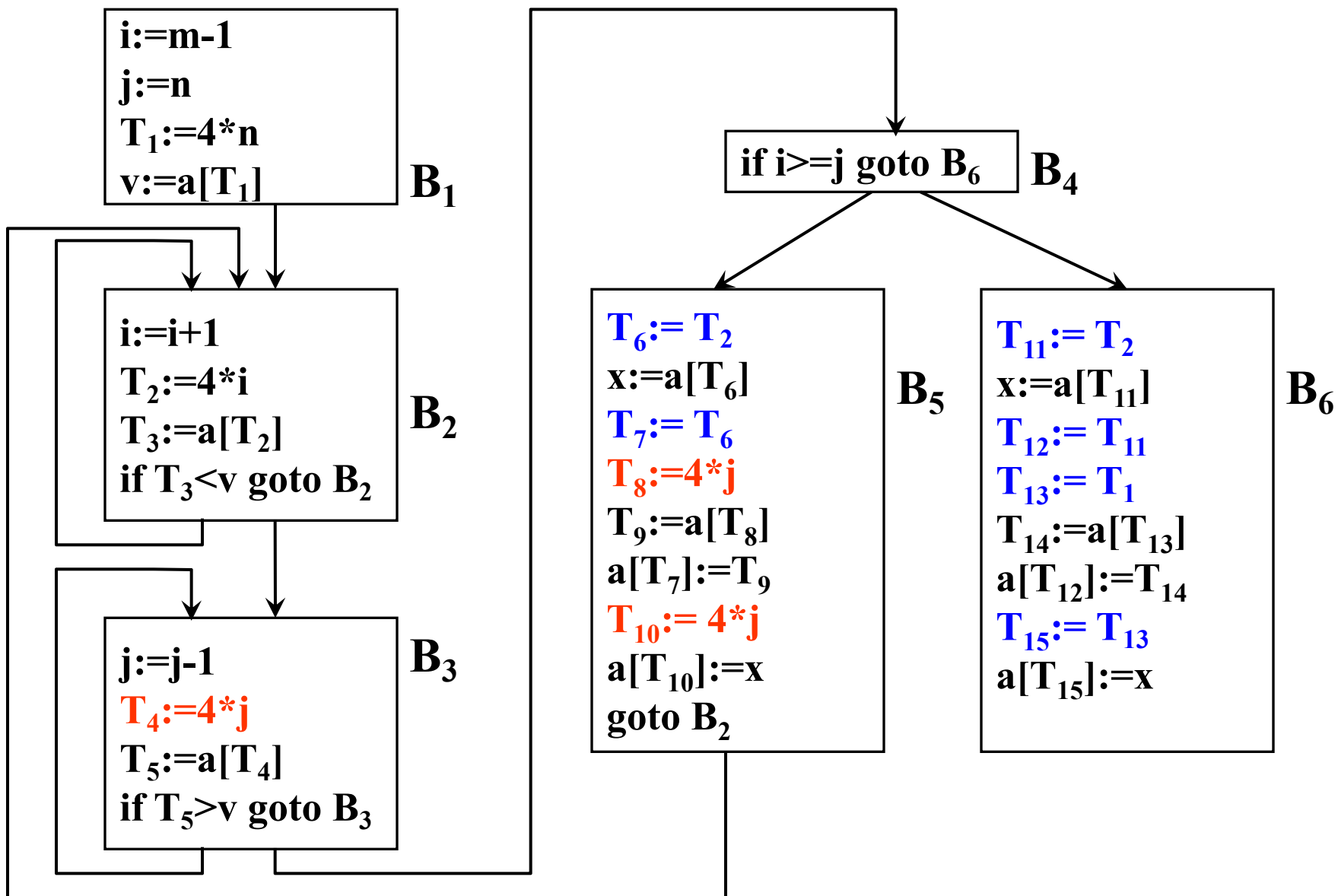


图10.3 删除公共子表达式

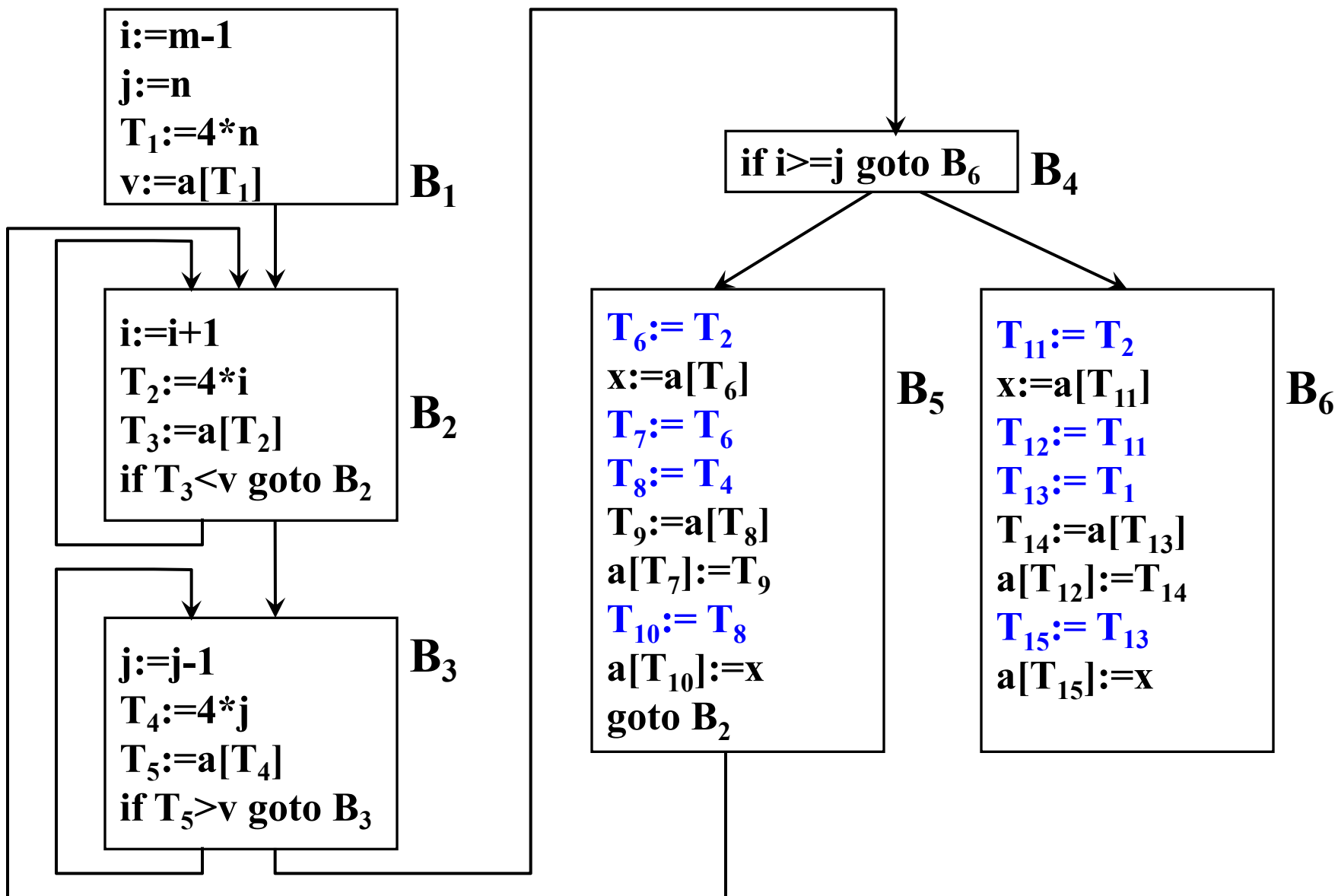


图10.3 删除公共子表达式后

- 复写传播

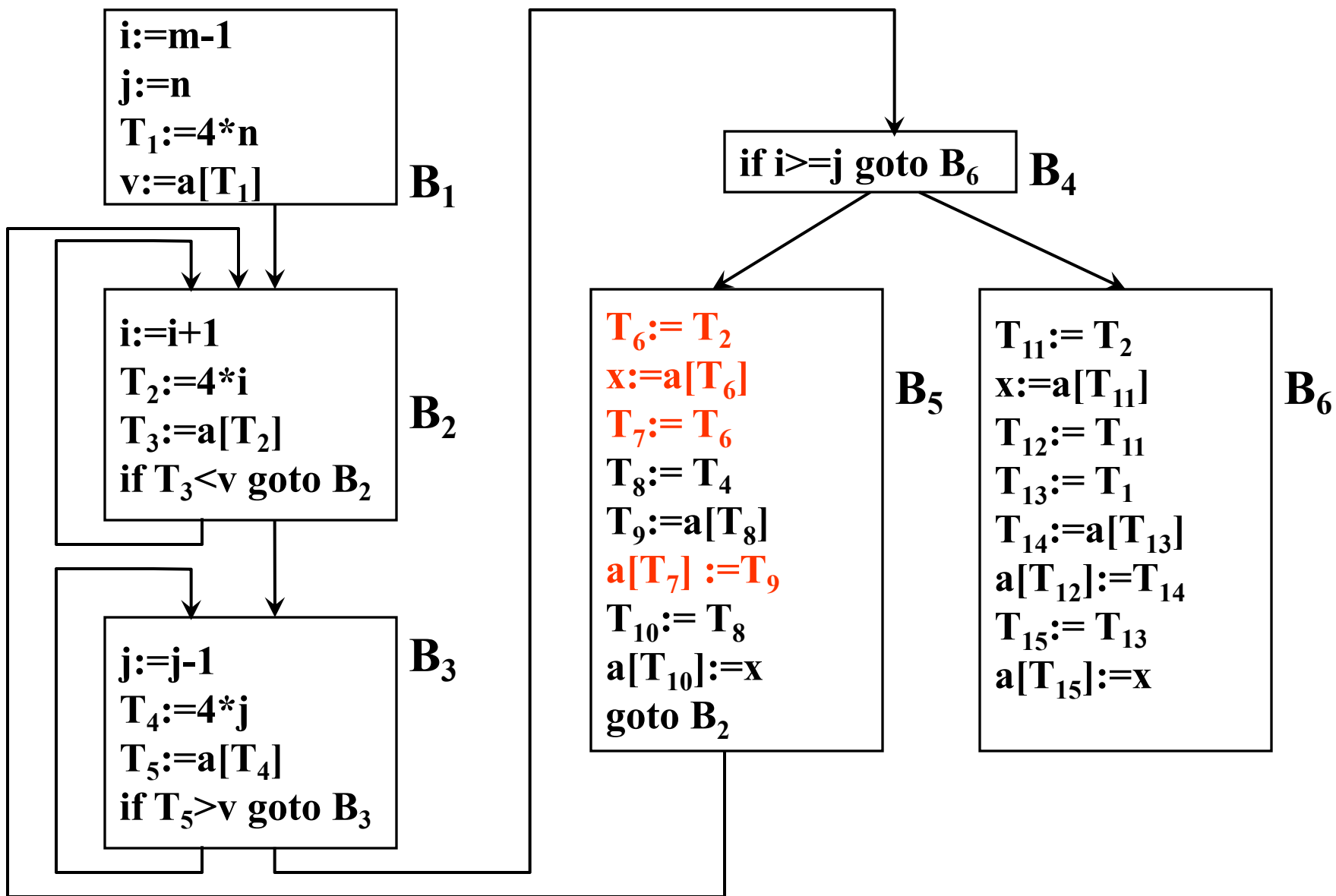


图10.4.1 复写传播

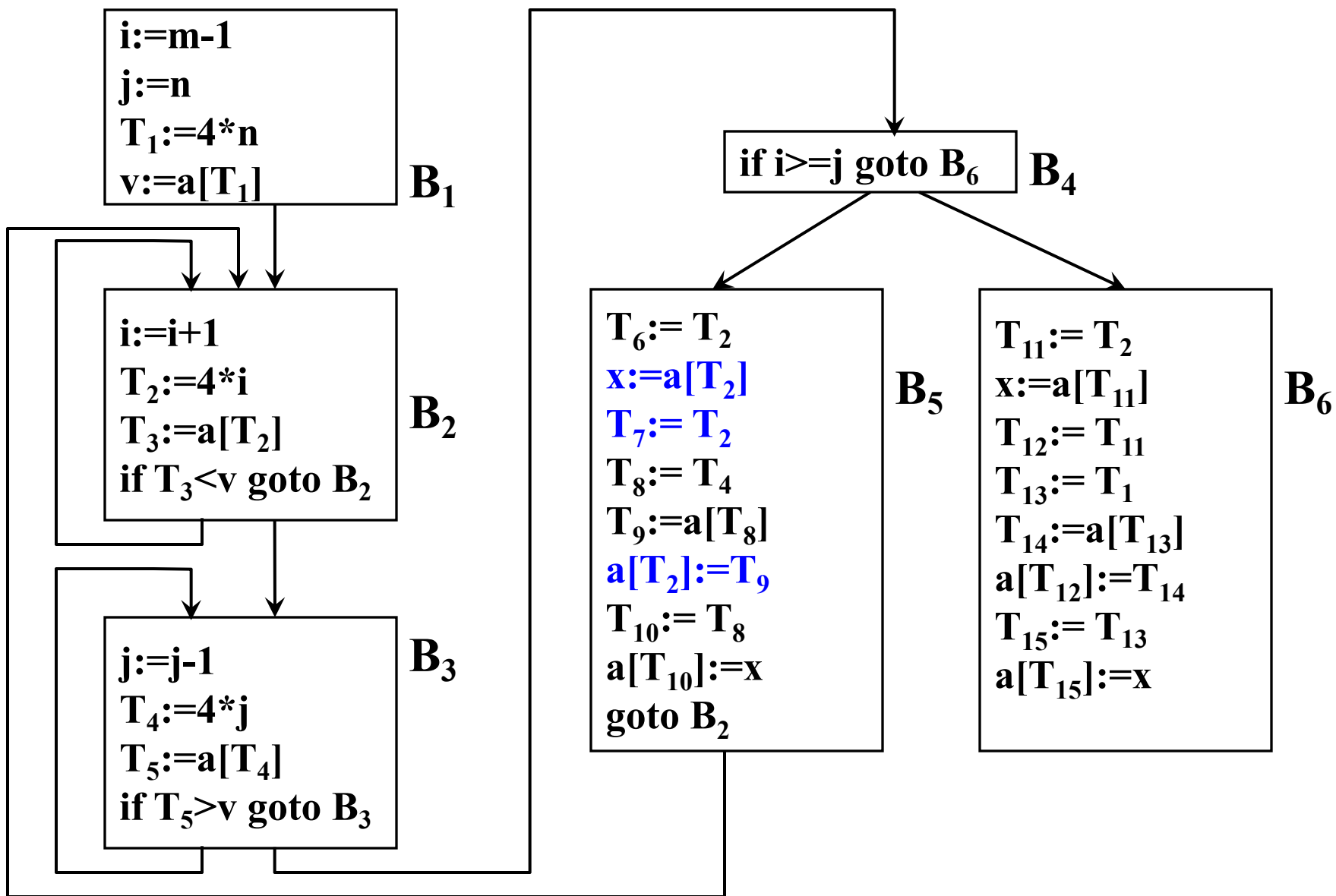


图10.4.1 复写传播

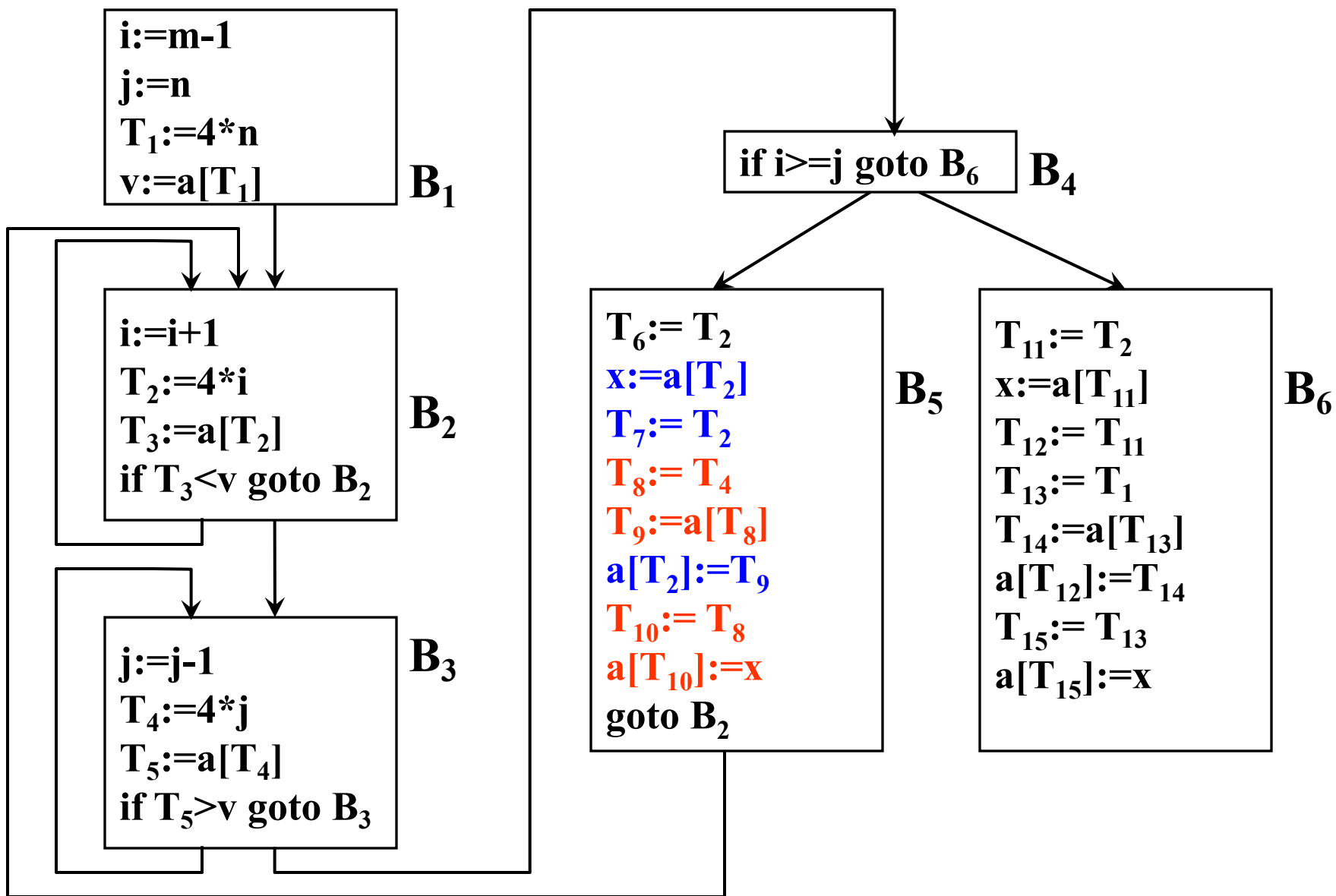


图10.4.1 复写传播

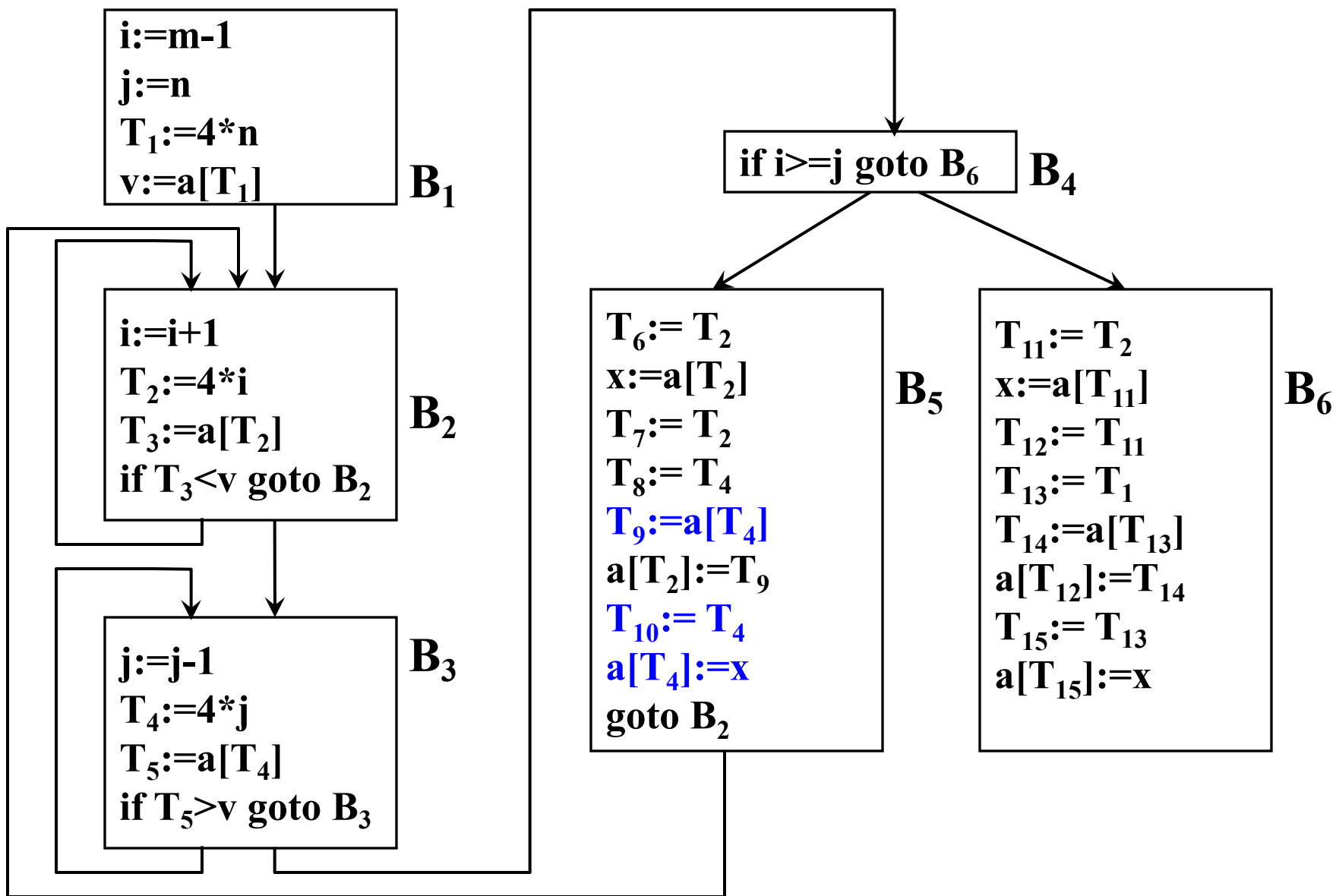


图10.4.1 复写传播

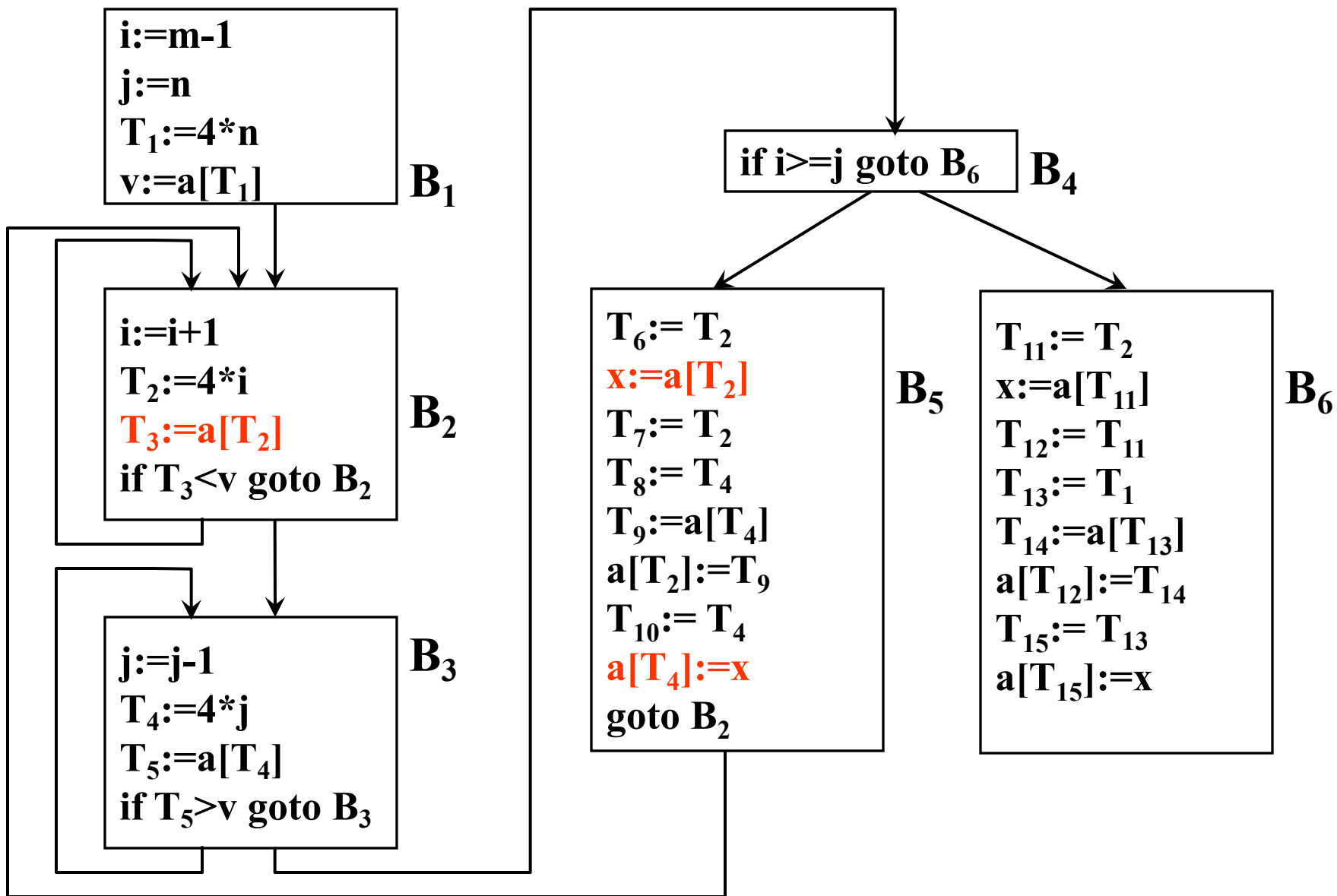


图10.4.1 复写传播

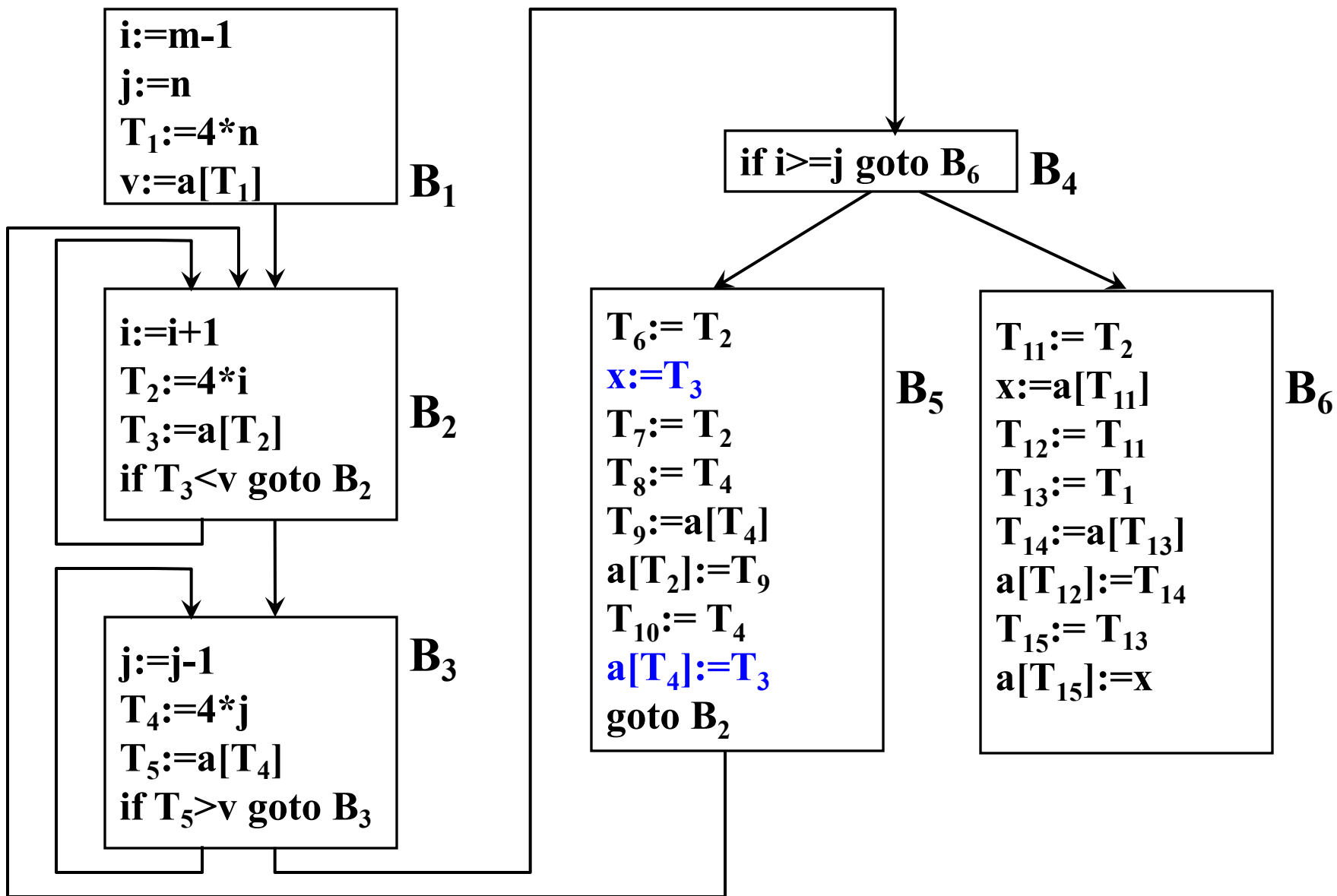


图10.4.1 复写传播

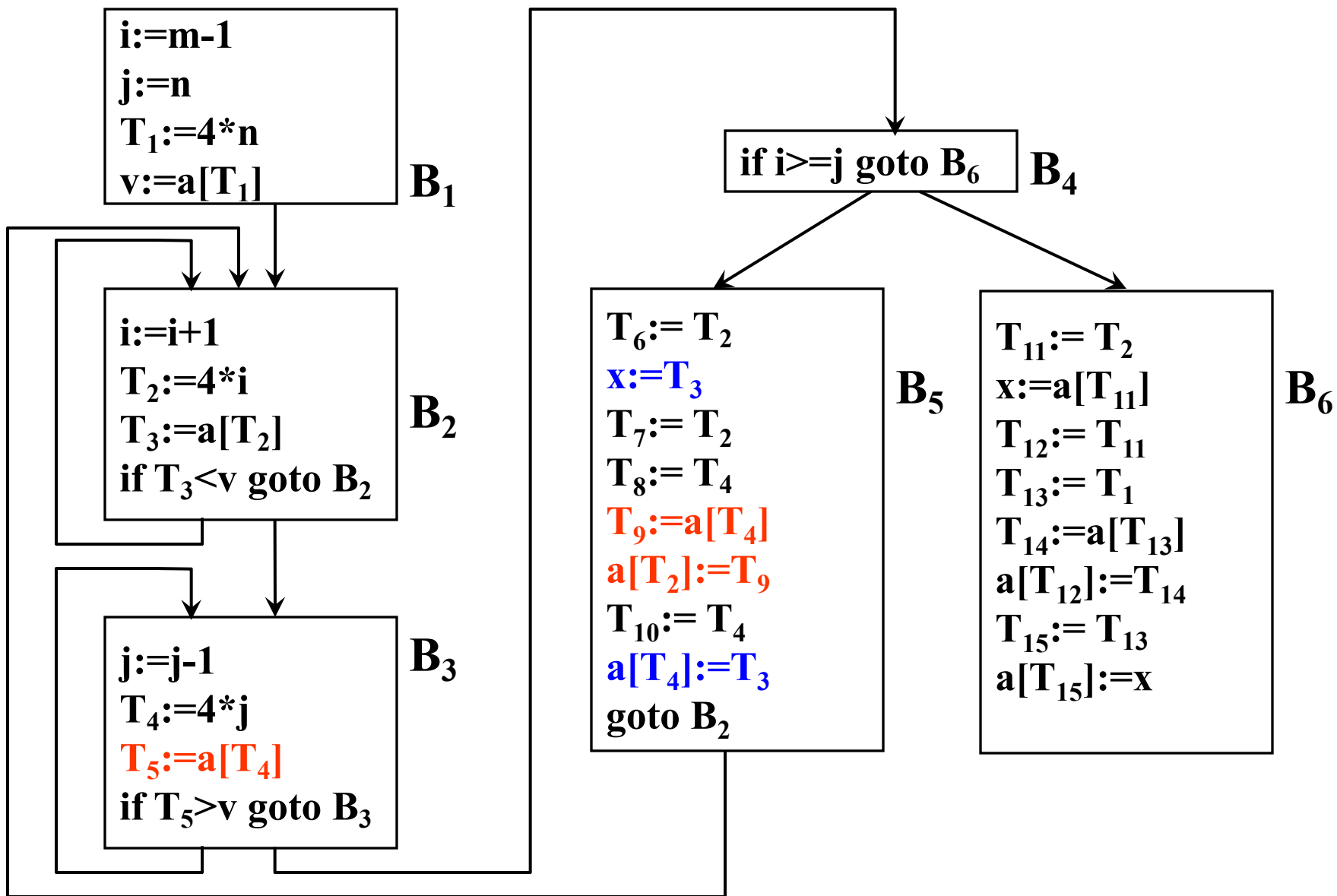


图10.4.1 复写传播

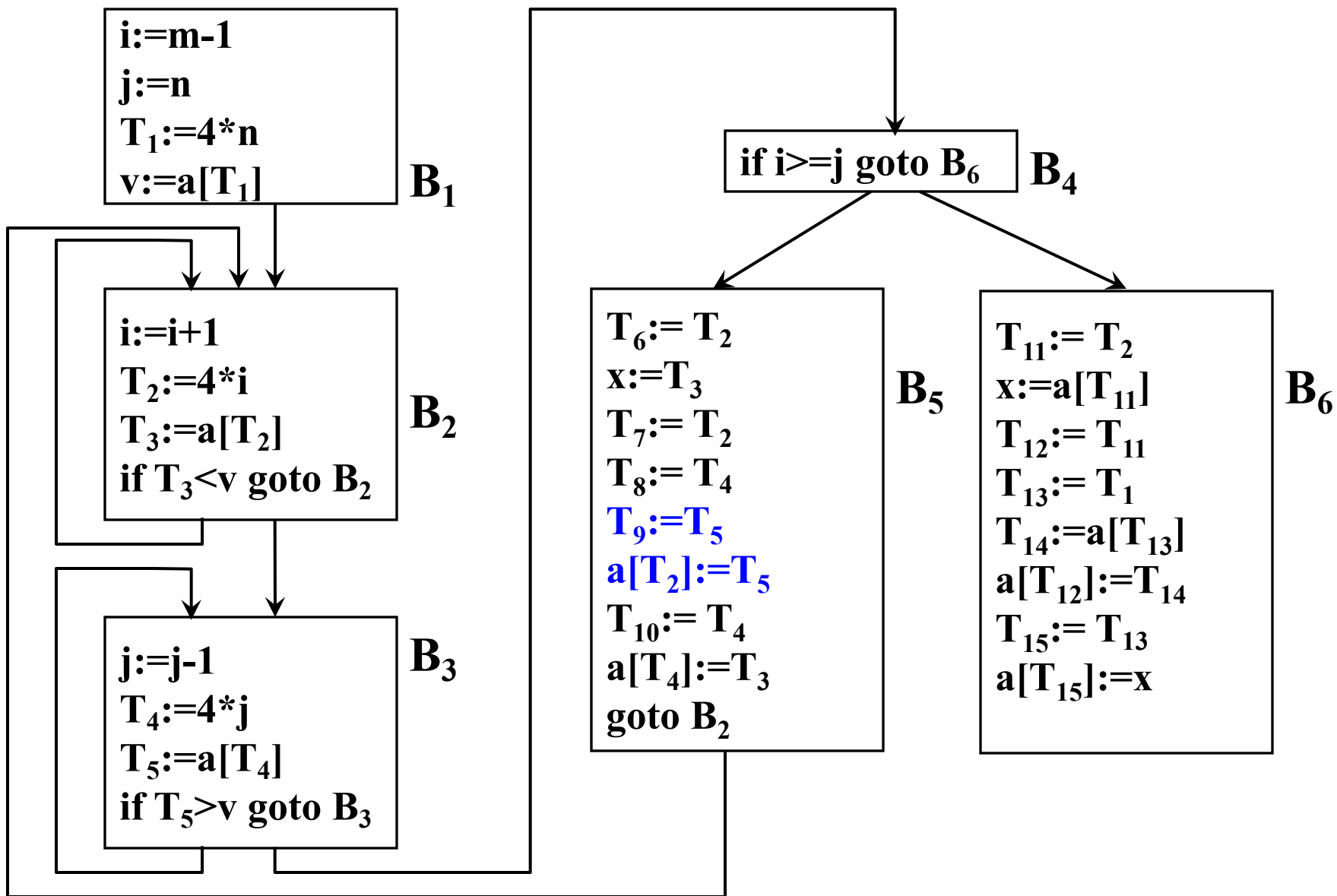


图10.4.1 复写传播

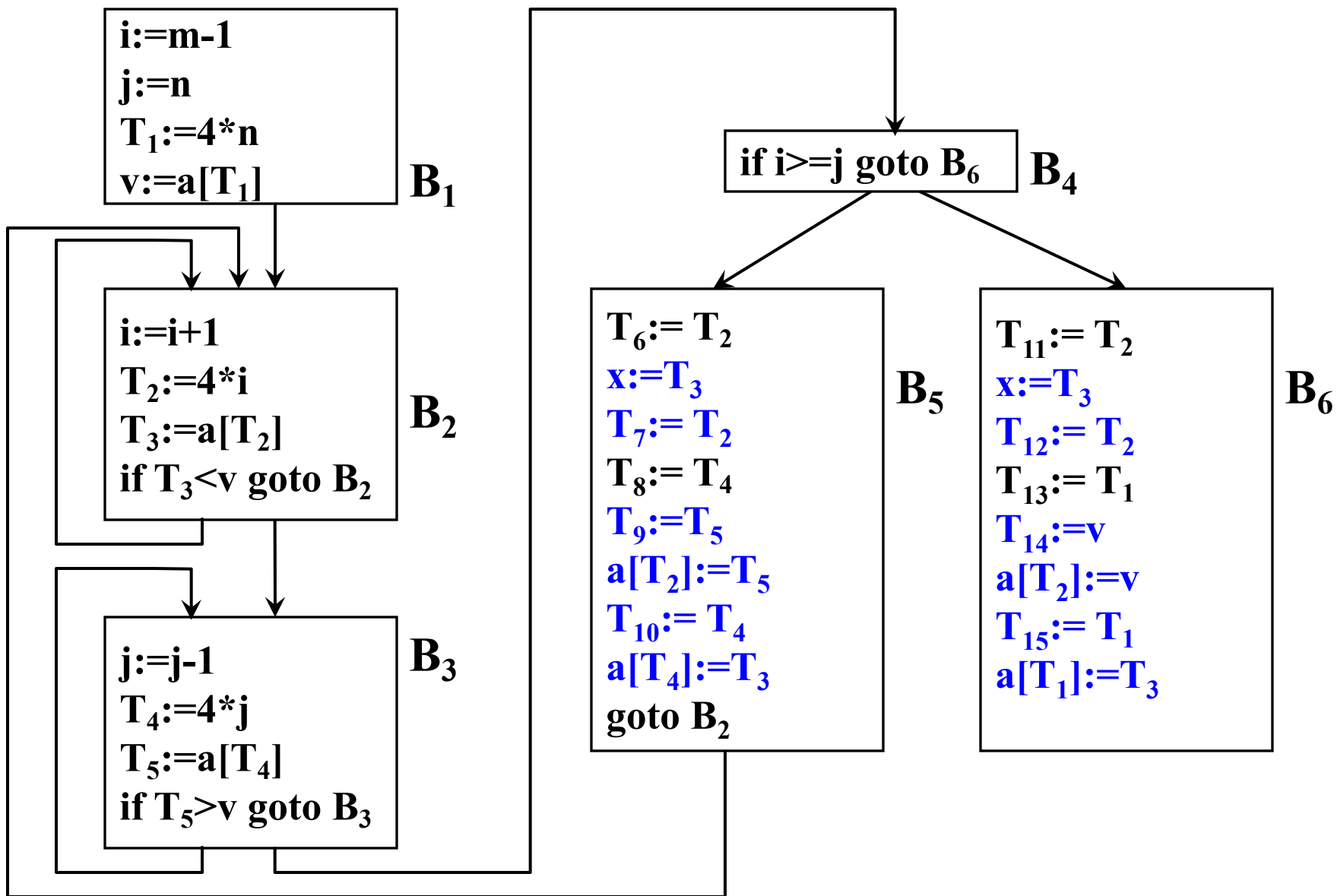


图10.4.1 复写传播后

- 复写传播
 - 复写传播的目的是使对某些变量的赋值变为无用。

- 删除无用代码/删除无用赋值

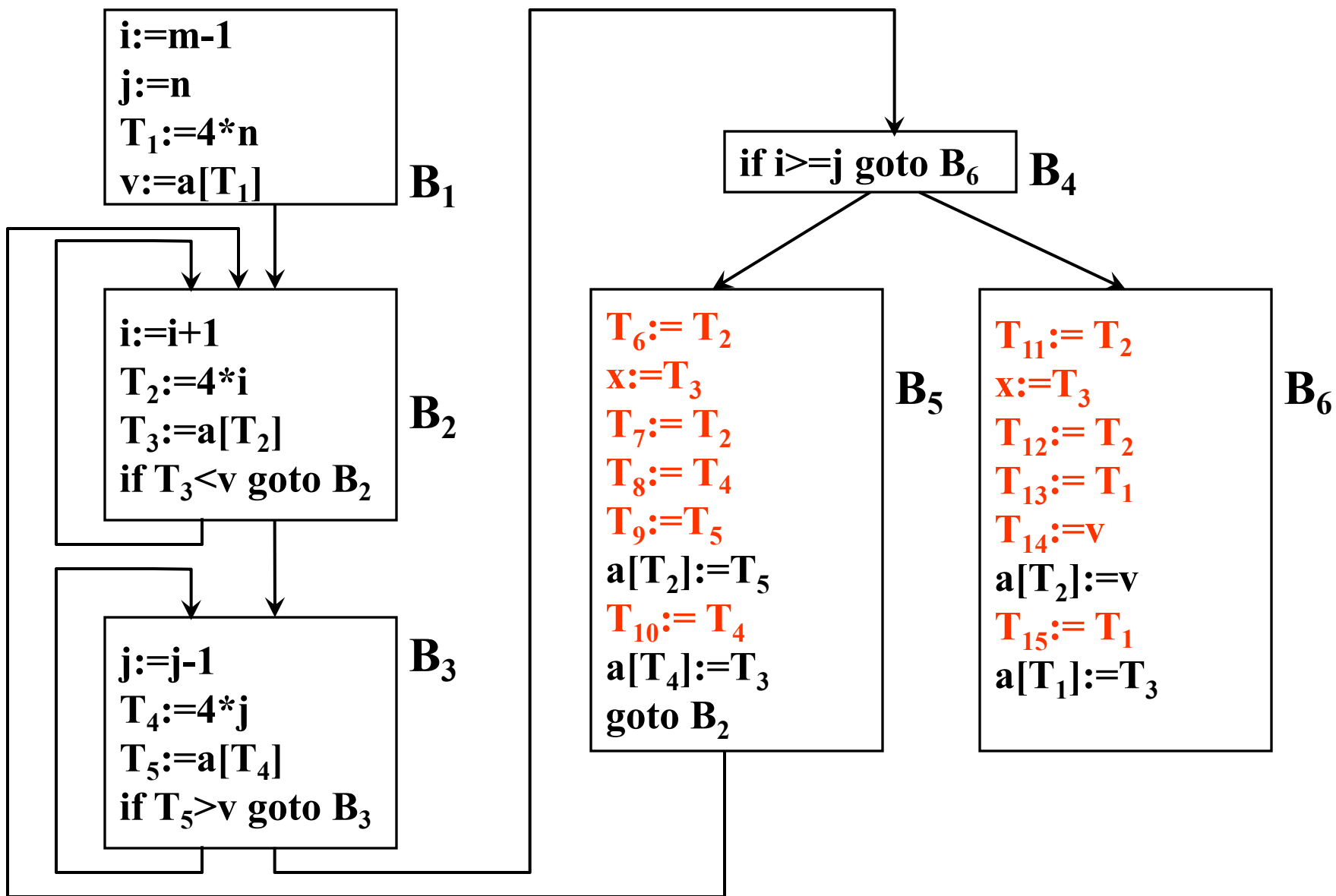


图10.4.2 删除无用赋值

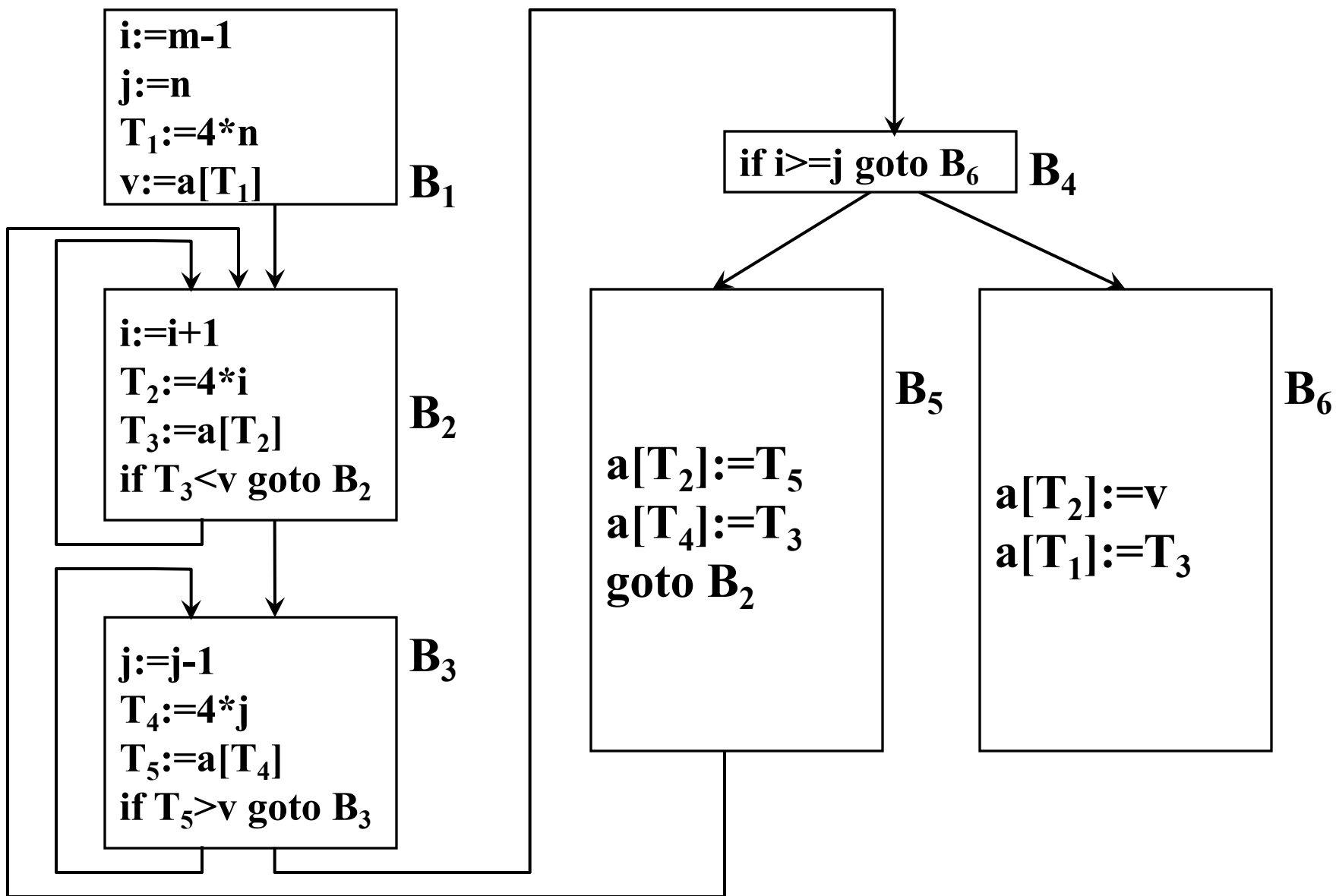


图10.4.2 删除无用赋值后

- 强度削弱

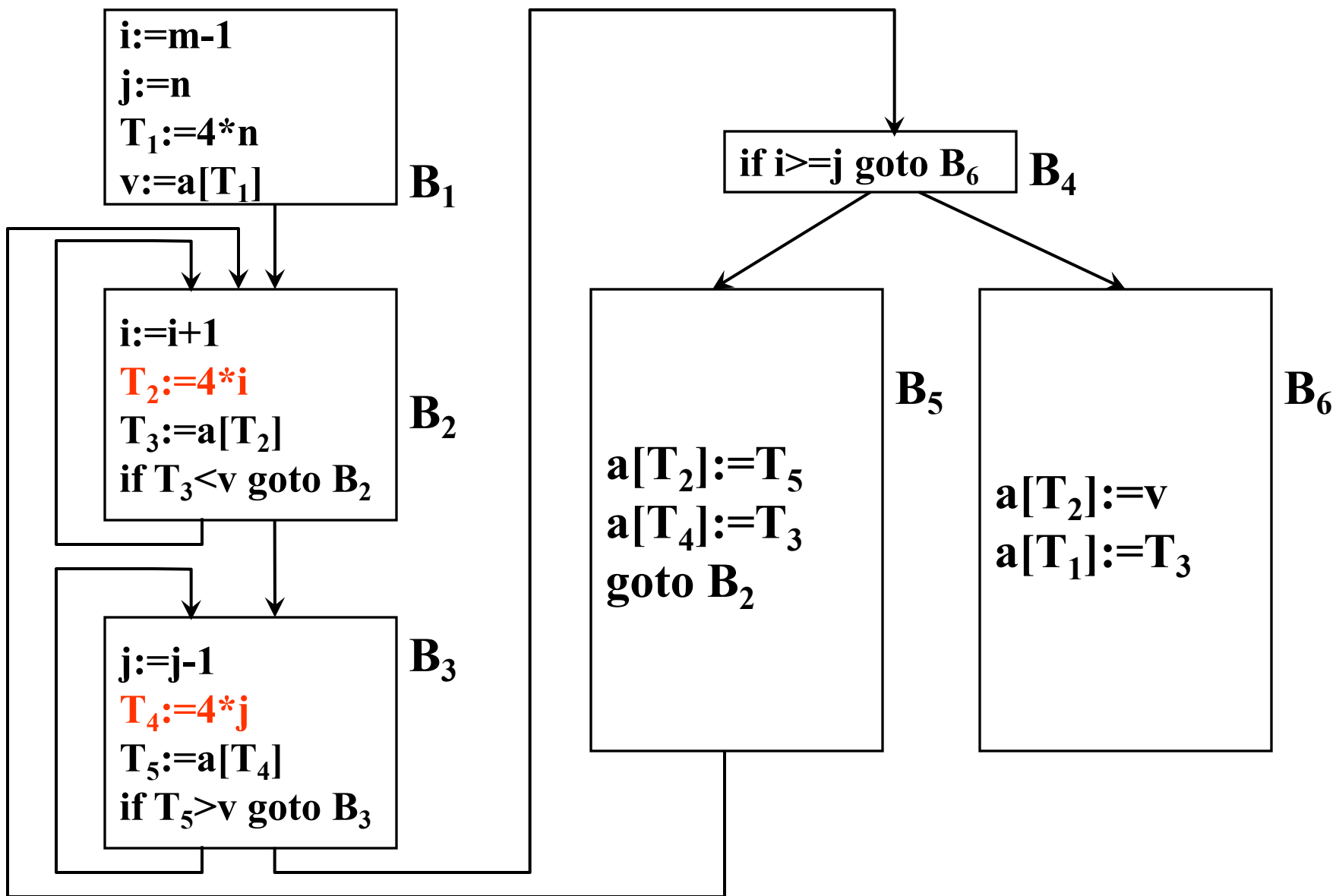


图10.5 强度削弱

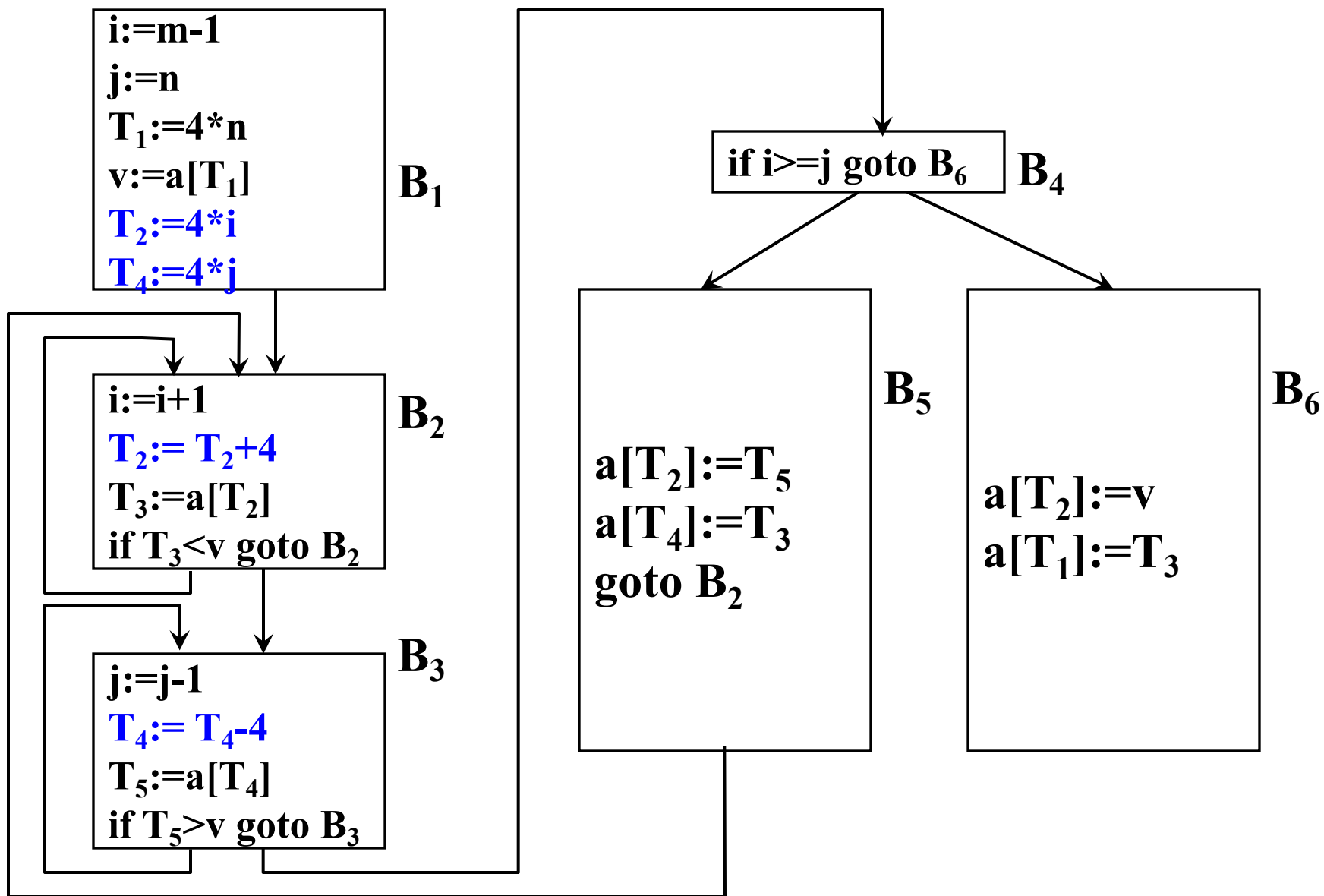


图10.5 强度削弱后

- 删除归纳变量

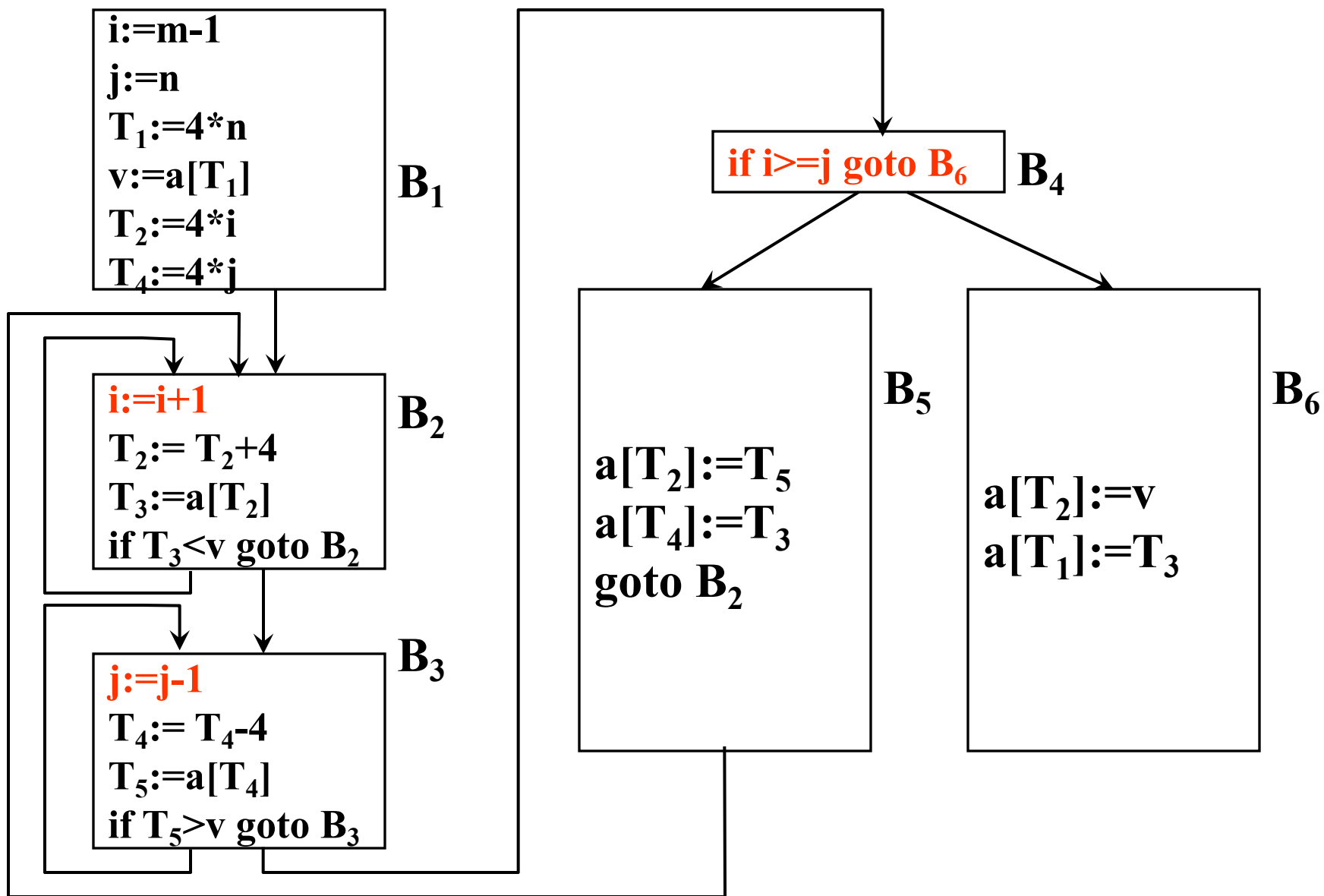


图10.6 删除归纳变量

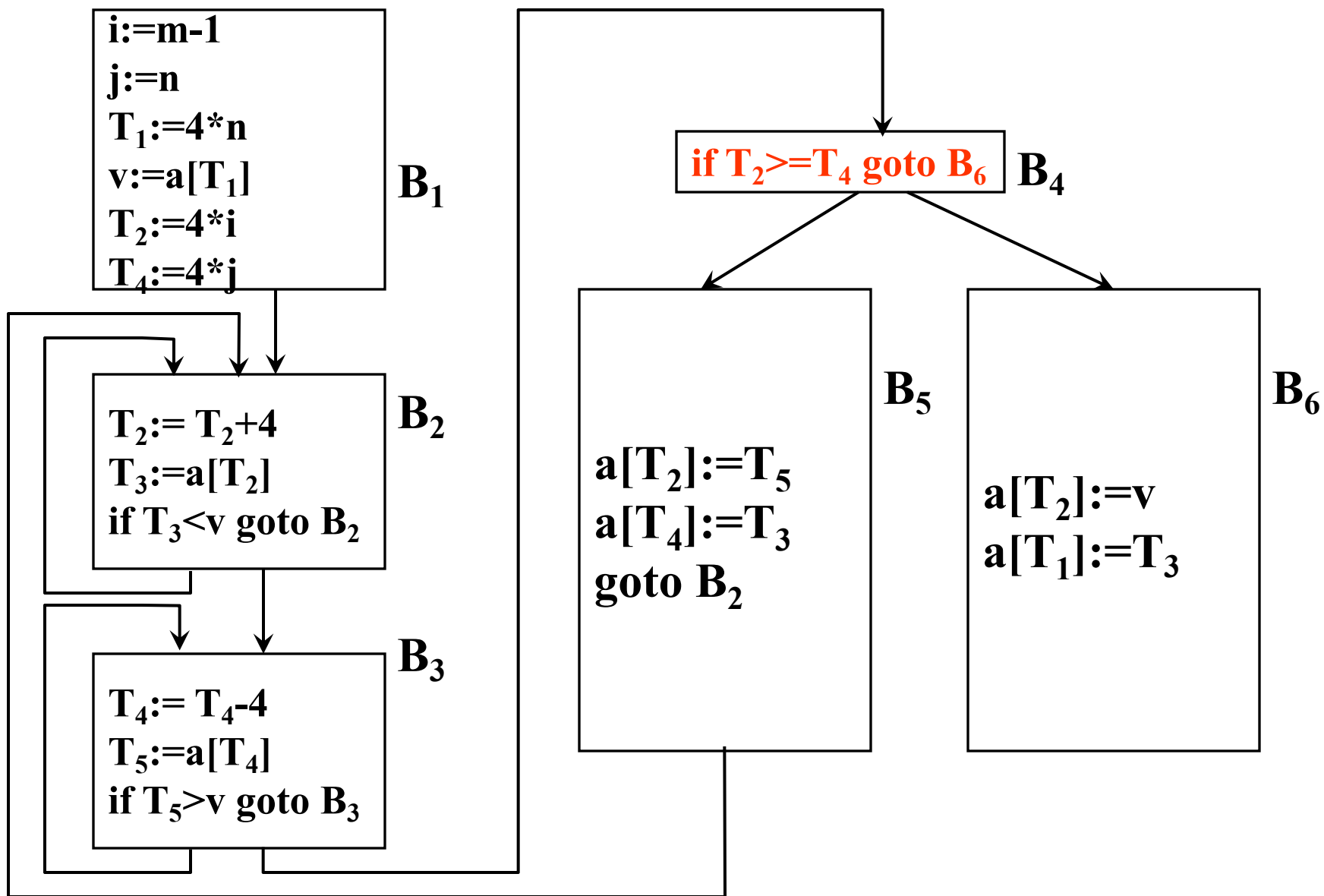


图10.6 删除归纳变量后

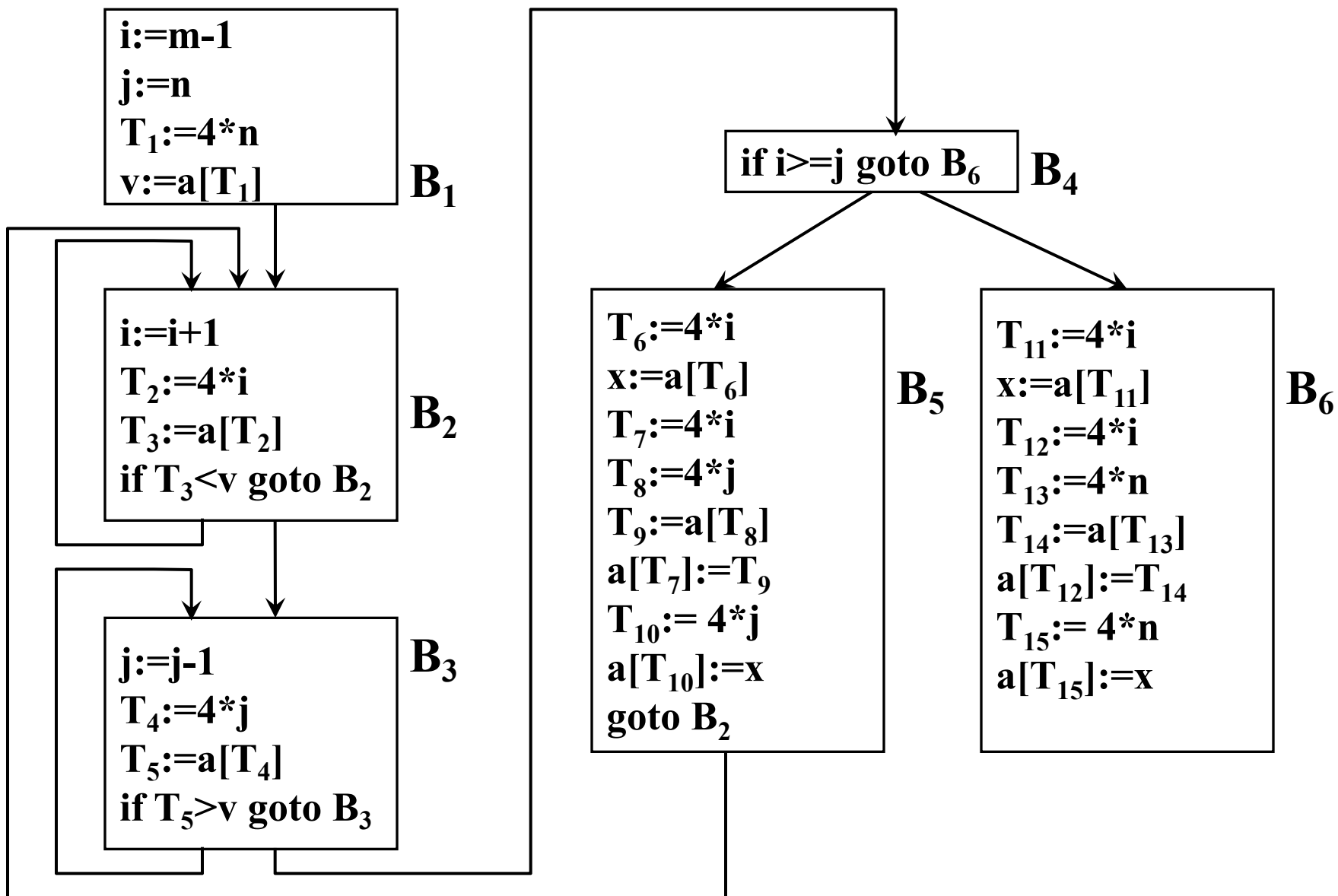


图10.2 中间代码程序段

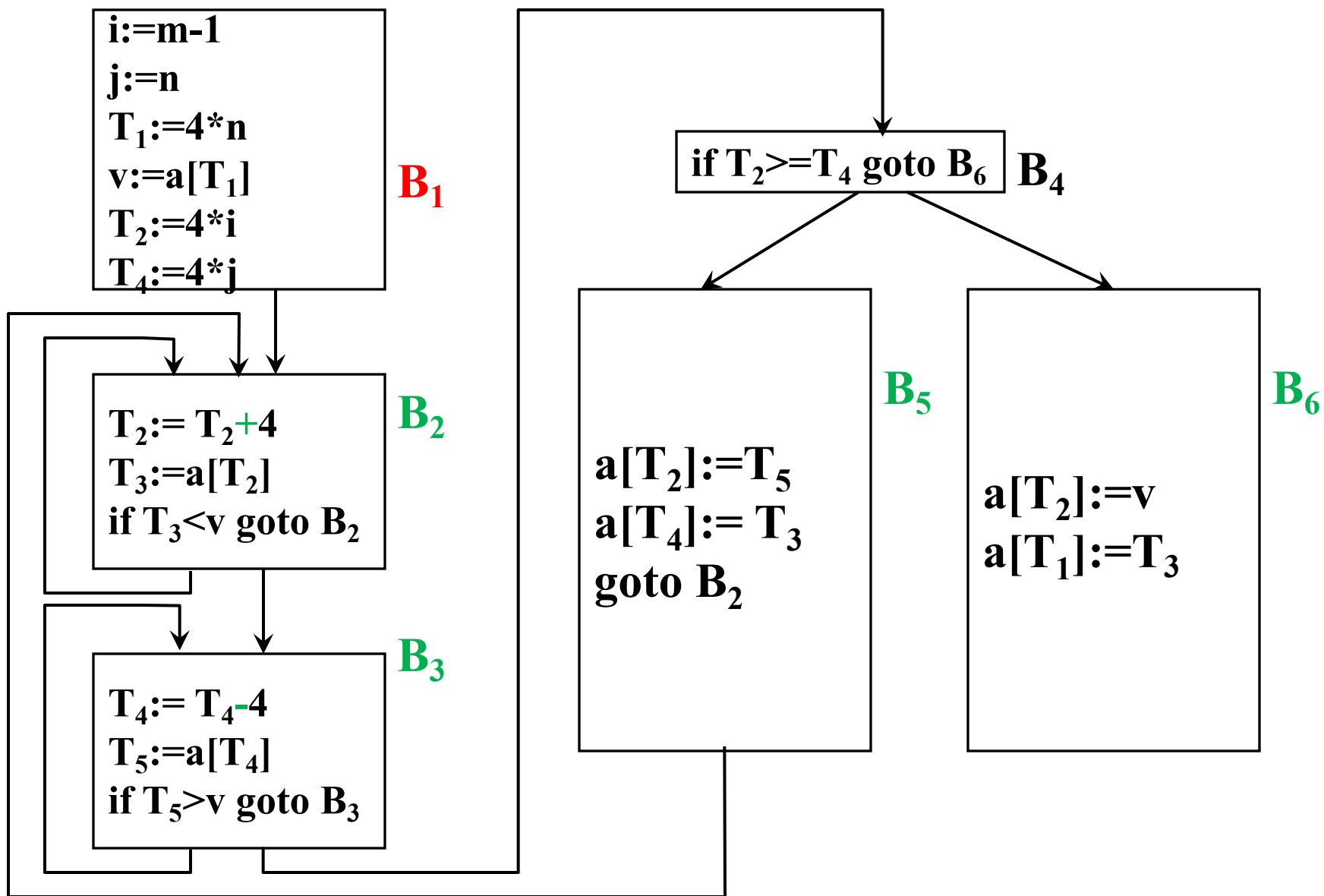


图10.6 删除归纳变量后

10.2 局部优化

10.2.1 基本块及流图

10.2.2 基本块的DAG表示及其应用

10.2.1 基本块及流图

- 基本块

指程序中一顺序执行的语句序列，其中只有一个入口和一个出口，入口是其中的第一个语句，出口是其中的最后一个语句。

执行时，只能从其入口进入，从其出口退出。

例，基本块：

$t_1 := a * a$

$t_2 := a * b$

$t_3 := 2 * t_2$

$t_4 := t_1 + t_3$

$t_5 := b * b$

$t_6 := t_4 + t_5$

10.2.1 基本块及流图

- 基本块

如果一条三地址语句为 $x := y + z$ ，则称对 x 定值并引用 y 和 z 。

在一个基本块中的一个名字，所谓在程序中的某个给定点是活跃的，是指如果在程序中（包括在本基本块或在其它基本块中）它的值在该点以后被引用。

例，基本块：

$t_1 := a * a$

$t_2 := a * b$

$t_3 := 2 * t_2$

$t_4 := t_1 + t_3$

$t_5 := b * b$

$t_6 := t_4 + t_5$

10.2.1 基本块及流图

- 局部优化

局限于基本块范围内的优化称为基本块内的优化，或称为局部优化。

10.2.1 基本块及流图

- 划分四元式程序为基本块的算法

1. 求出四元式程序中各个基本块的入口语句：

- (1) 程序的第一个语句，或者
- (2) 能由条件转移语句或无条件转移语句转移到的语句，或者
- (3) 紧跟在条件转移语句后面的语句。

2. 对以上求出的每一入口语句，构造其所属的基本块：由该入口语句到另一入口语句（不包括该入口语句），或到一停语句（包括该停语句）之间的语句序列组成。

3. 凡未被纳入某一基本块中的语句，可把它们从程序中删除。

- 例10.1 划分以下三地址代码程序为基本块

- (1) read X
- (2) read Y
- (3) $R := X \bmod Y$
- (4) if $R = 0$ goto (8)
- (5) $X := Y$
- (6) $Y := R$
- (7) goto (3)
- (8) write Y
- (9) halt

1. 求出四元式程序中各个基本块的入口语句：

- (1) 程序的第一个语句，或者
- (2) 能由条件转移语句或无条件转移语句转移到的语句，或者
- (3) 紧跟在条件转移语句后面的语句。

- 例10.1 划分以下三地址代码程序为基本块

(1) read X

(2) read Y

(3) $R := X \bmod Y$

(4) if $R = 0$ goto (8)

(5) $X := Y$

(6) $Y := R$

(7) goto (3)

(8) write Y

(9) halt

1. 求出四元式程序中各个基本块的入口语句：

(1) 程序的第一个语句，或者

(2) 能由条件转移语句或无条件转移语句转移到的语句，或者

(3) 紧跟在条件转移语句后面的语句。

- 例10.1 划分以下三地址代码程序为基本块

(1) read X

(2) read Y

(3) $R := X \bmod Y$

(4) if $R = 0$ goto (8)

(5) $X := Y$

(6) $Y := R$

(7) goto (3)

(8) write Y

(9) halt

1. 求出四元式程序中各个基本块的入口语句：

(1) 程序的第一个语句，或者

(2) 能由条件转移语句或无条件转移语句转移到的语句，或者

(3) 紧跟在条件转移语句后面的语句。

- 例10.1 划分以下三地址代码程序为基本块

(1) read X

(2) read Y

(3) $R := X \bmod Y$

(4) if $R = 0$ goto (8)

(5) $X := Y$

(6) $Y := R$

(7) goto (3)

(8) write Y

(9) halt

1. 求出四元式程序中各个基本块的入口语句：

(1) 程序的第一个语句，或者

(2) 能由条件转移语句或无条件转移语句转移到的语句，或者

(3) 紧跟在条件转移语句后面的语句。

- 例10.1 划分以下三地址代码程序为基本块

(1) read X

(2) read Y

(3) $R := X \bmod Y$

(4) if $R = 0$ goto (8)

(5) $X := Y$

(6) $Y := R$

(7) goto (3)

(8) write Y

(9) halt

2. 对以上求出的每一入口语句，构造其所属的基本块：由该入口语句到另一入口语句（不包括该入口语句），或到一停语句（包括该停语句）之间的语句序列组成。

- 例10.1 划分以下三地址代码程序为基本块

(1) read X

(2) read Y

(3) $R := X \bmod Y$

(4) if $R = 0$ goto (8)

(5) $X := Y$

(6) $Y := R$

(7) goto (3)

(8) write Y

(9) halt

3. 凡未被纳入某一基本块中的语句，可把它们从程序中删除。

10.2.1 基本块及流图

- 流图
 - 将控制流的信息增加到基本块的集合上来表示一个程序
 - 每个流图以基本块为结点
 - 如果一个结点的基本块的入口语句是程序的第一条语句，则称此结点为首结点

10.2.1 基本块及流图

- 流图

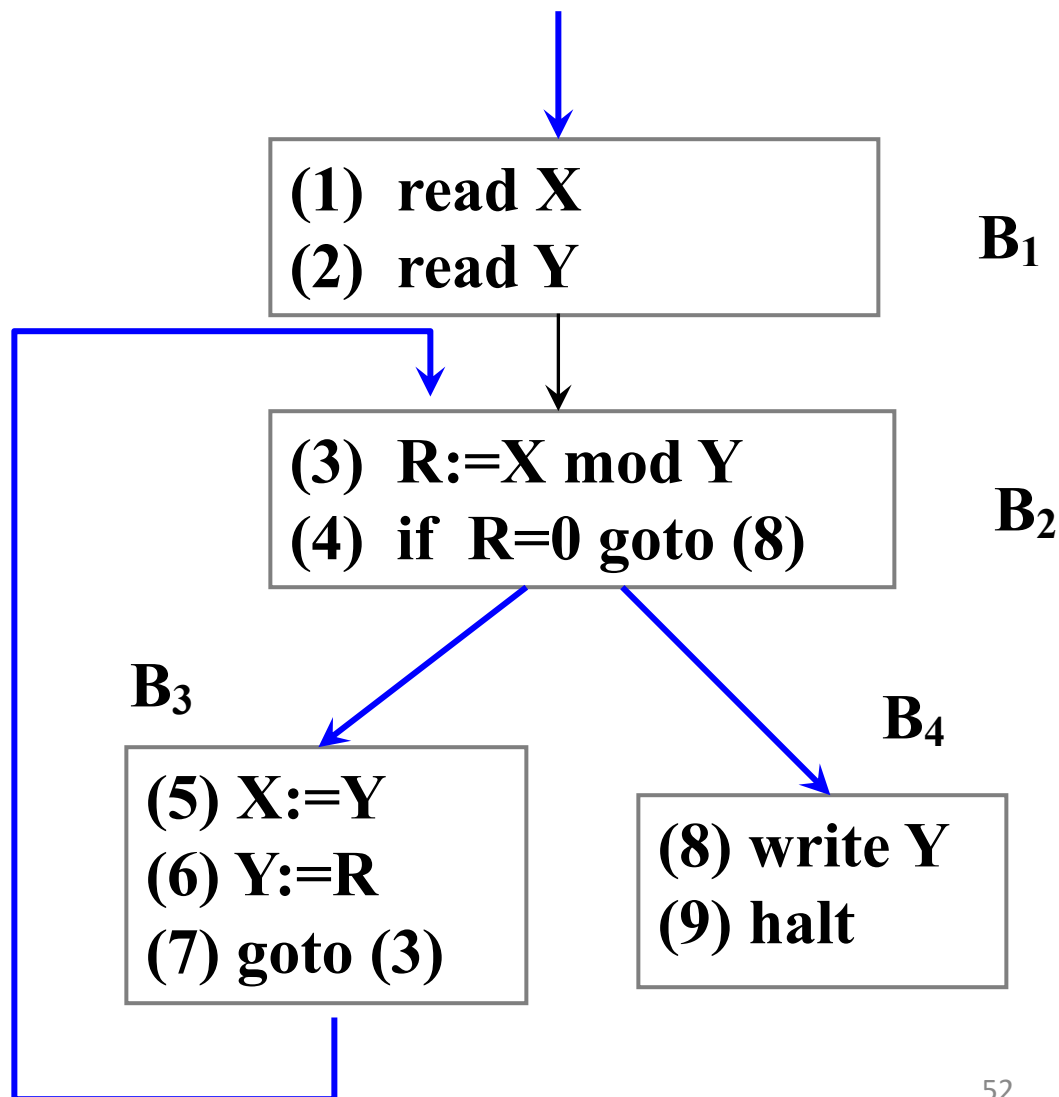
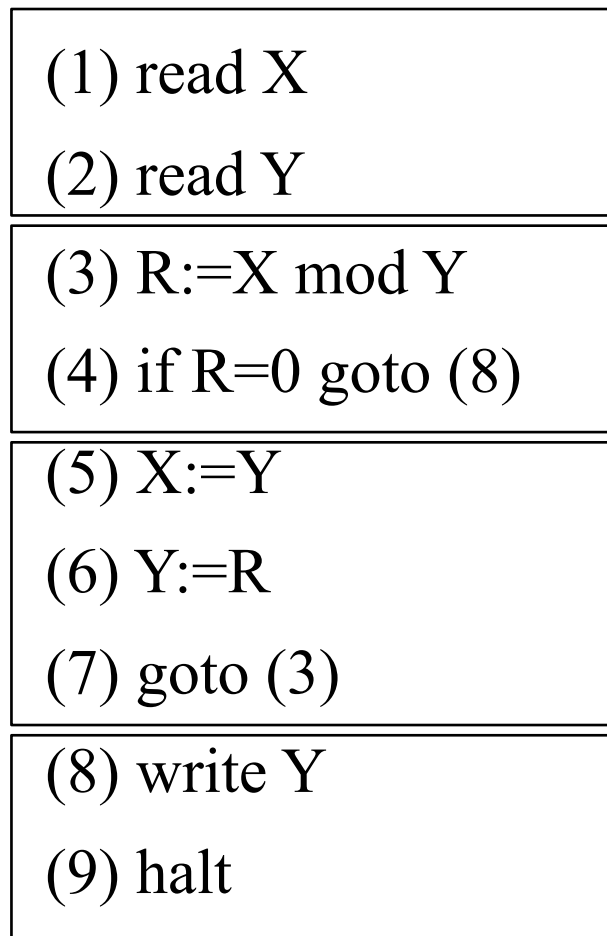
- 如果在某个执行顺序中，基本块 B_2 紧接在基本块 B_1 之后执行，即，如果

- (1) 有一个条件或无条件转移语句从 B_1 的最后一条语句转移到 B_2 的第一条语句；或

- (2) 在程序的序列中， B_2 紧接在 B_1 的后面，并且 B_1 的最后一条语句不是一个无条件转移语句。

- 则从 B_1 到 B_2 有一条有向边， B_1 是 B_2 的前驱， B_2 是 B_1 的后继。

10.2.1 基本块及流图



10.2.1 基本块及流图

(1) read X

(2) read Y

(3) $R := X \bmod Y$

(4) if $R=0$ goto (8)

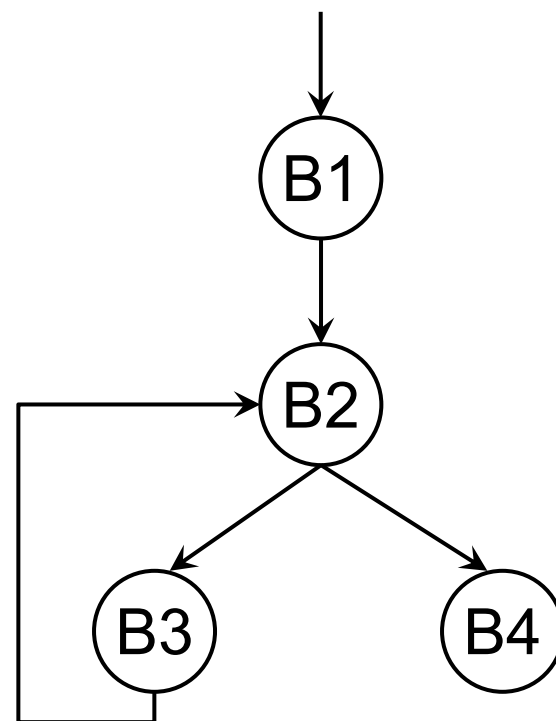
(5) $X := Y$

(6) $Y := R$

(7) goto (3)

(8) write Y

(9) halt



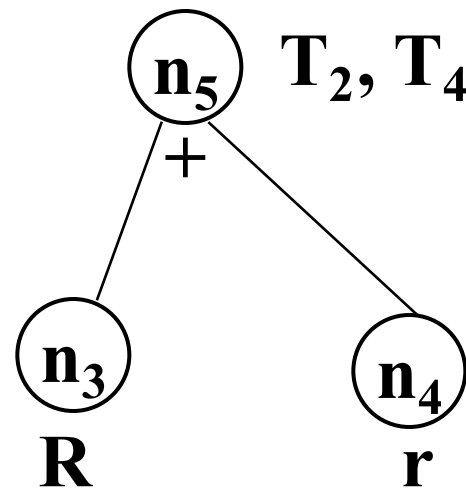
10.2.2 基本块的DAG表示及其应用

• 描述计算过程的DAG是一种带有下述标记或附加信息的DAG:

- 图的**叶结点**以一**标识符**或**常数**作为标记, 表示该结点代表该变量或常数的值
- 图的**内部结点**以一**运算符**作为标记, 表示该结点代表应用该运算符对其后继结点所代表的值进行运算的结果
- 各个结点上可能**附加一个或多个标识符**(称附加标识符)表示这些变量具有该结点所代表的值



3.14



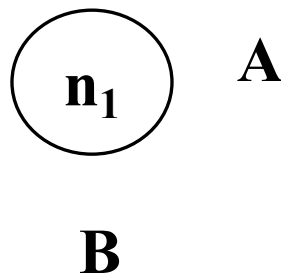
10.2.2 基本块的DAG表示及应用

- 一个基本块，可用一个DAG来表示
与各四元式相对应的DAG结点形式：

四元式

DAG 图

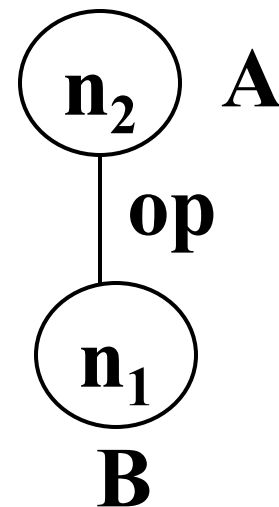
(0) 0型: $A := B$
($:=$, B , $-$, A)



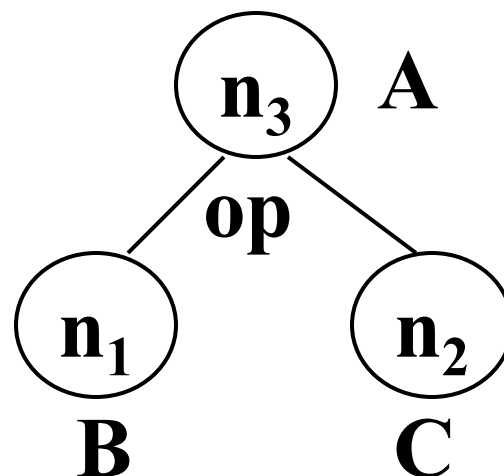
四元式

DAG 图

(1) 1型: $A := \text{op } B$
($\text{op}, B, -, A$)



(2) 2型: $A := B \text{ op } C$
(op, B, C, A)



- 假设**DAG**各结点信息将用某种适当的数据结构存放(如链表)。
- 标识符与结点的对应函数NODE(A)：如果存在一个结点n, A是其上的标记或附加标识符, 则返回n, 否则返回null
- 中间代码形式：
 - (0) $A := B$
 - (1) $A := \text{op } B$
 - (2) $A := B \text{ op } C$
- 构造DAG算法
 - 1. 初始DAG为空, 无任何结点
 - 2. 依次对基本块中的每一条中间代码执行3—5操作

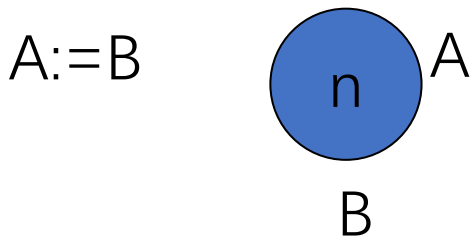
- 3. $A := B$

(1) 如果 $\text{NODE}(B)$ 无定义，则构造一标记为 B 的叶结点并定义 $\text{NODE}(B)$ 为这个结点。

记 $\text{NODE}(B)$ 的值为 n 。

(2) 如果 $\text{NODE}(A)$ 无定义，则把 A 附加在结点 n 上并令 $\text{NODE}(A) = n$ ；

否则先把 A 从 $\text{NODE}(A)$ 结点上的附加标识符集中删除（即删除无用赋值），把 A 附加到新结点 n 上并令 $\text{NODE}(A) = n$ 。



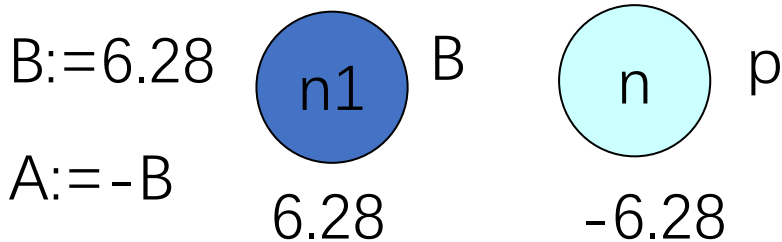
- 4. $A := \text{op } B$

(1) 如果 $\text{NODE}(B)$ 无定义, 则构造一标记为 B 的叶结点并定义 $\text{NODE}(B)$ 为这个结点。

- 4. $A := op\ B$

(2) 如果 $NODE(B)$ 是标记为常数的叶结点,

- ① 执行 $op\ B$ (即 **合并已知量**), 令得到的新常数为 p 。
- ② 如果 $NODE(B)$ 是处理当前代码时新构造出来的结点, 则删除它。
- ③ 如果 $NODE(p)$ 无定义, 则构造一用 p 做标记的叶结点 n ;
- ④ 置 $NODE(p) = n$ 。



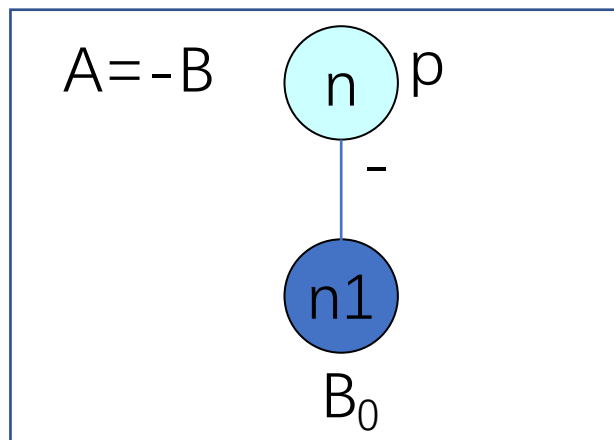
- 4. $A := op\ B$

(2) 如果 $NODE(B)$ 是标记为常数的叶结点,

- ① 执行 $op\ B$ (即合并已知量), 令得到的新常数为 p 。
- ② 如果 $NODE(B)$ 是处理当前代码时新构造出来的结点, 则删除它。
- ③ 如果 $NODE(p)$ 无定义, 则构造一用 p 做标记的叶结点 n
- ④ 置 $NODE(p)=n$ 。

否则,

- ① 检查DAG中是否已有一结点, 其唯一后继为 $NODE(B)$ 且标记为 op (即公共子表达式)。
- ② 如果没有, 则构造该结点 n , 否则就把已有的结点作为它的结点并设该结点为 n 。



- 4. $A := \text{op } B$

(3) 如果 $\text{NODE}(A)$ 无定义，则把 A 附加在结点 n 上并令 $\text{NODE}(A) = n$;

否则先把 A 从 $\text{NODE}(A)$ 结点上的附加标识符集中删除（即删除无用赋值），把 A 附加到新结点 n 上并令 $\text{NODE}(A) = n$ 。

- 5. $A := B \text{ op } C$

(1) 如果 $\text{NODE}(B)$ 无定义，则构造一标记为B的叶结点并定义 $\text{NODE}(B)$ 为这个结点。

如果 $\text{NODE}(C)$ 无定义，则构造一标记为C的叶结点并定义 $\text{NODE}(C)$ 为这个结点。

- 5. $A := B \text{ op } C$

(2) 如果 $\text{NODE}(B)$ 和 $\text{NODE}(C)$ 都是标记为常数的叶结点,

- ① 执行 $B \text{ op } C$ (即合并已知量), 令得到的新常数为 p 。
- ② 如果 $\text{NODE}(B)$ 或 $\text{NODE}(C)$ 是处理当前代码时新构造出来的结点, 则删除它。
- ③ 如果 $\text{NODE}(p)$ 无定义, 则构造一用 p 做标记的叶结点 n ;
- ④ 置 $\text{NODE}(p) = n$ 。

否则,

- ① 检查DAG中是否已有一结点, 其左后继为 $\text{NODE}(B)$, 右后继为 $\text{NODE}(C)$, 且标记为 op (即公共子表达式)。
- ② 如果没有, 则构造该结点 n , 否则就把已有的结点作为它的结点并设该结点为 n 。

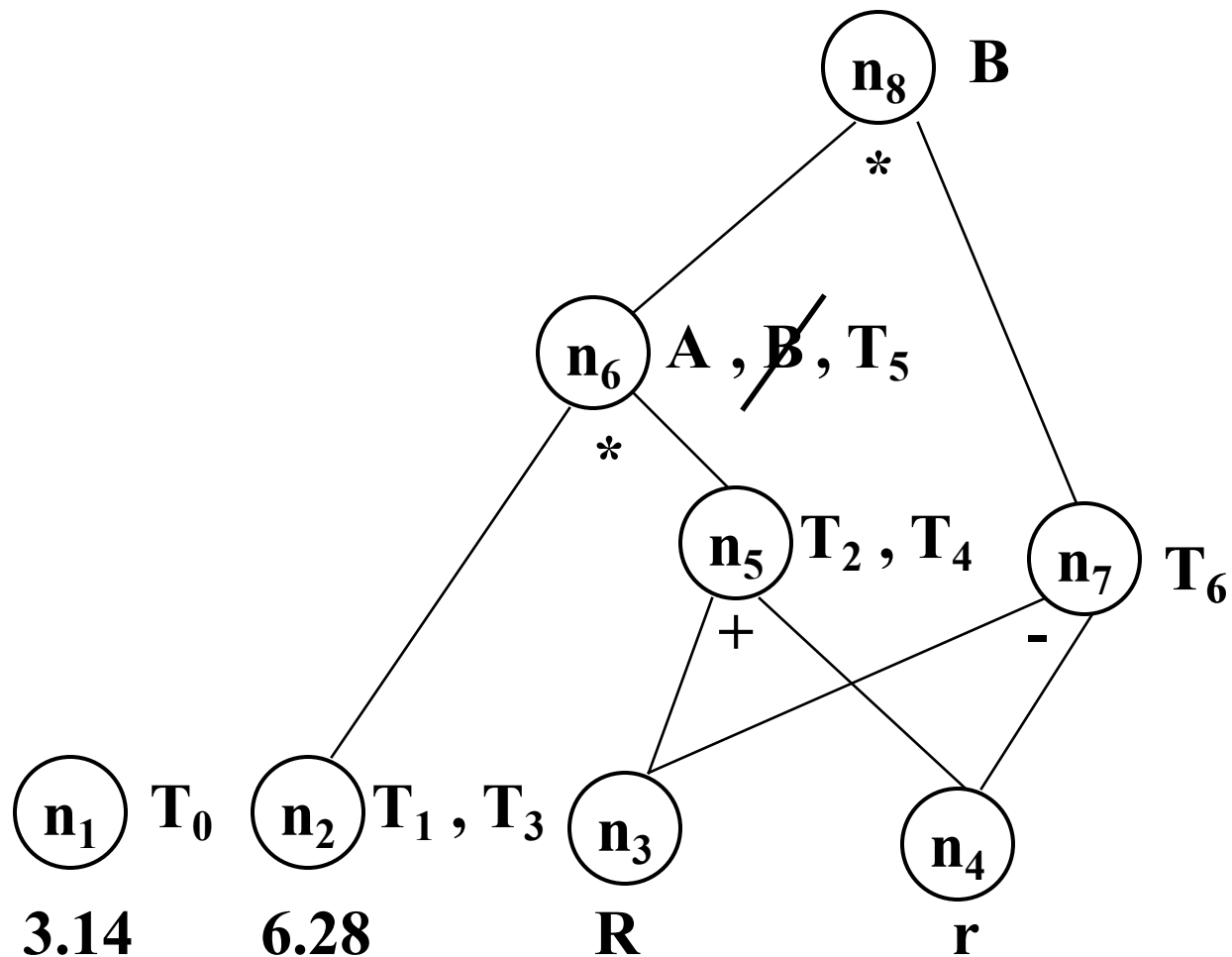
- 5. $A := B \text{ op } C$

(3) 如果 $\text{NODE}(A)$ 无定义, 则把 A 附加在结点 n 上并令 $\text{NODE}(A) = n$;

否则先把 A 从 $\text{NODE}(A)$ 结点上的附加标识符集中删除 (即 **删除无用赋值**) , 把 A 附加到新结点 n 上并令 $\text{NODE}(A) = n$ 。

例10.4 试构造以下基本块G的DAG

- (1) $T_0 := 3.14$
- (2) $T_1 := 2 * T_0$
- (3) $T_2 := R + r$
- (4) $A := T_1 * T_2$
- (5) $B := A$
- (6) $T_3 := 2 * T_0$
- (7) $T_4 := R + r$
- (8) $T_5 := T_3 * T_4$
- (9) $T_6 := R - r$
- (10) $B := T_5 * T_6$



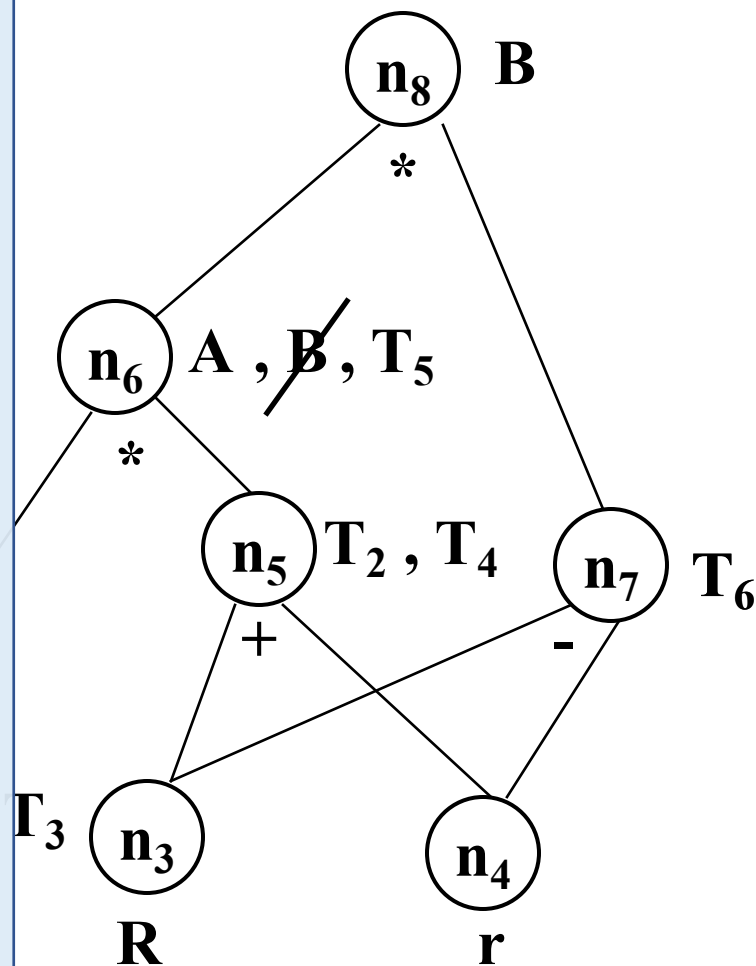
例10.4 试构造以下基本块G的DAG

优化前:

- (1) $T_0 := 3.14$
- (2) $T_1 := 2 * T_0$
- (3) $T_2 := R + r$
- (4) $A := T_1 * T_2$
- (5) $B := A$
- (6) $T_3 := 2 * T_0$
- (7) $T_4 := R + r$
- (8) $T_5 := T_3 * T_4$
- (9) $T_6 := R - r$
- (10) $B := T_5 * T_6$

优化后:

- (1) $T_0 := 3.14$
- (2) $T_1 := 6.28$
- (3) $T_3 := 6.28$
- (4) $T_2 := R + r$
- (5) $T_4 := T_2$
- (6) $A := 6.28 * T_2$
- (7) $T_5 := A$
- (8) $T_6 := R - r$
- (9) $B := A * T_6$



例10.4 试构造以下基本块G的DAG

优化前G:

- (1) $T_0 := 3.14$
- (2) $T_1 := 2 * T_0$
- (3) $T_2 := R + r$
- (4) $A := T_1 * T_2$
- (5) $B := A$
- (6) $T_3 := 2 * T_0$
- (7) $T_4 := R + r$
- (8) $T_5 := T_3 * T_4$
- (9) $T_6 := R - r$
- (10) $B := T_5 * T_6$

无用赋值

优化后G':

- (1) $T_0 := 3.14$
- (2) $T_1 := 6.28$
- (3) $T_3 := 6.28$
- (4) $T_2 := R + r$
- (5) $T_4 := T_2$
- (6) $A := 6.28 * T_2$
- (7) $T_5 := A$
- (8) $T_6 := R - r$
- (9) $B := A * T_6$

合并已知量

删除公共子表达式